

北京工业大学

2025- 2026 学年 第 1 学期

计算机学院

计算机科学与技术（实验班）

课程名称：	数据结构课程设计		
报告性质：	课程设计报告		
学号：	23071005	姓名：	侯景祺
任课教师：	杜永萍	课程性质：	必修
班级：	230710	成绩：	
教师评语：			

2025 年 12 月 16 日

摘要

数据结构与算法是计算机科学的核心基础，但其逻辑抽象、过程复杂的特点往往给学习者带来理解上的困难。为了将抽象的逻辑过程具象化，本课程设计开发了一款功能完善的“数据结构可视化演示系统”。

本系统基于 Python 3.12 语言开发，采用 PySide6 框架构建现代化的图形用户界面，并严格遵循 Model-View-Controller 软件架构模式，实现了数据逻辑与视图渲染的解耦。

在功能模块方面，系统主要包含两大核心部分：

数据结构可视化：支持顺序表、链表、栈等线性结构，以及二叉搜索树（BST）、AVL 自平衡树、哈夫曼树、B-树等复杂树形结构的动态演示。系统能够实时绘制节点的插入、删除、查找路径以及树的旋转与分裂过程。

算法动画演示：内置了 0/1 背包问题、表达式求值（中缀转后缀及计算）、括号匹配等经典算法。通过 Python 生成器技术，系统将连续的算法执行过程转化为可控的“帧”，实现了“单步调试”级的可视化，清晰展示了回溯过程、栈状态变化及数据流动。

此外，本系统创新性地集成了 AI 智能助手模块（基于大语言模型接口，Gemma 3n），实现了自然语言交互功能。用户可以通过输入如“建立一个由 10, 20, 5 组成的 BST”等指令，直接控制系统进行绘图与操作，极大地提升了系统的交互性与智能化水平。

测试结果表明，该系统运行稳定、界面美观、交互逻辑清晰，能够准确、生动地展示数据结构与算法的内部机理，有效辅助了相关知识的学习与理解。

关键词：数据结构可视化；PySide6；MVC 架构；算法动画；AI 智能交互；Python

目录

- 摘要2
- 1 需求分析.....5
 - 1.1 程序功能.....5
 - 1.1.1 基本功能.....5
 - 1.1.2 扩展功能.....5
 - 1.1.3 隐含功能.....6
 - 1.2 需要处理的数据.....6
 - 1.2.1 后端.....6
 - 1.2.2 前端.....7
 - 1.3 开发环境.....7
 - 1.4 用户界面设计.....7
- 2. 数据结构演示设计.....9
 - 2.1 主要数据结构.....9
 - 2.1.1 线性表与栈.....9
 - 2.1.2 树形结构.....10
 - 2.2 程序架构.....11
 - 2.2.1 整体架构.....11
 - 2.2.2 核心模块关系.....11
 - 2.2.3 线性结构模块.....12
 - 2.2.4 二叉树模块.....12
 - 2.2.5 高级树形结构模块.....13
 - 2.2.6 AI 智能交互模块.....14
- 3. 算法演示设计.....14
 - 3.1 主要算法.....14
 - 3.1.1 背包问题（基于栈的回溯）.....15
 - 3.1.2 栈的应用算法.....15
 - 3.2 程序架构.....15
 - 3.2.1 整体架构.....15
 - 3.2.2 核心模块关系.....16
- 4. 详细设计.....16
 - 4.1 数据结构核心算法.....16
 - 4.1.1 AVL 树插入与自平衡 (AVLTree.insert)16
 - 4.1.2 B-树节点分裂 (BTree.split_child)20
 - 4.2 可视化系统逻辑设计.....21
 - 4.2.1 基于生成器的分步演示机制.....21
 - 4.3 扩展功能设计.....23
 - 4.3.1 AI 智能指令解析.....23
- 5. 测试.....29
 - 5.1 正常运行测试.....30
 - 5.1.1 线性结构操作测试.....30
 - 5.1.2 树形结构构建与操作测试.....30
 - 5.1.3 AVL 树自平衡测试.....33

5.1.4	哈夫曼树与 B-树测试.....	33
5.1.5	普通二叉树演示测试.....	35
5.1.6	算法动态演示测试.....	37
5.1.7	括号匹配问题测试.....	41
5.1.8	AI 智能交互测试.....	42
5.1.9	数据结构保存、读取测试.....	43
5.2	错误用例.....	44
5.2.1	非法输入测试.....	44
5.2.2	逻辑违规操作.....	45
5.3	极端情况测试.....	46
5.3.1	空数据处理.....	46
5.3.2	复杂边界情况.....	46
6.	总结与提高.....	47
6.1	体会与收获.....	47
6.2	问题与解决.....	48
6.3	自我评估.....	48

1 需求分析

1.1 程序功能

本系统旨在构建一个交互式的数据结构与算法可视化演示平台，帮助用户通过图形化界面直观理解抽象的计算机科学概念。系统集成了多种经典数据结构和算法，支持动态操作、分步演示以及智能化的交互方式。

1.1.1 基本功能

基本功能涵盖了数据结构课程中最基础的线性表与树形结构，重点在于展示其核心操作逻辑。

模块类别	数据结构	功能描述
线性结构	顺序表 (Sequential List)	支持在指定索引位置插入和删除元素，能够动态展示数组元素移动 (Shift) 的过程。
	链表 (Linked List)	演示单链表的节点插入（重点展示指针重连过程）和删除操作，直观展示节点间的链接关系变化。
	栈 (Stack)	模拟栈的“后进先出” (LIFO) 特性，提供入栈 (Push)、出栈 (Pop) 和查看栈顶 (Top) 的可视化操作，动态展示栈的生长与消退。
树形结构	二叉搜索树 (BST)	支持动态插入节点构建树，提供查找路径高亮与删除操作的可视化。删除操作能演示寻找后继节点及替换的过程。
	哈夫曼树 (Huffman Tree)	根据用户输入的字符及其权重，演示基于优先队列的哈夫曼树构建过程，并生成哈夫曼编码。

1.1.2 扩展功能

扩展功能包括了更复杂的高级数据结构、经典算法的动态演示，以及提升用户体验的智能化交互与系统增强功能。

类别	功能点	详细描述
高级树形结构	AVL 树 (平衡二叉树)	在 BST 基础上增加了自平衡机制。演示插入/删除后，通过左旋 (LL/RR) 或 双旋 (LR/RL) 维持平衡的过程，实时显示平衡因子。
	B-树 (B-Tree)	演示多路搜索树的插入操作，重点展示节点关键字达到阈值 $(2t - 1)$ 时的节点分裂 (Split) 及中间关键字上移过程，支持自定义阶数 t 。
算法演示	背包问题 (Knapsack)	使用基于栈的回溯法演示 0/1 背包问题的求解过程，清晰展示尝试放入、容量判断、回溯以及寻找解的完整路径。
	表达式求值	包含“中缀转后缀”和“后缀表达式求值”两个子模块。演示运算符栈的优先级处理逻辑以及后缀表达式的计算过程。
	括号匹配	演示利用栈检测字符串中括号（圆括号、方括号、花括号）是否成对匹配的动态过程。

系统交互与增强	AI 智能助手	集成大语言模型（LLM）接口，支持自然语言交互。用户输入如“建一个 10, 20, 5 的 BST”指令，系统自动解析并执行绘图。
	分步演示控制	利用 Python 生成器技术，将算法执行转化为可控的“帧”，提供下一步、上一步、重置按钮，实现“单步调试”级可视化。
	通用二叉树构建	支持通过遍历序列还原二叉树。用户输入“前序+中序”、“后序+中序”或“层序”序列，系统自动构建对应结构。
	文件存取	支持将当前画布上的数据结构状态保存为 JSON 文件，并能从文件加载恢复，实现数据的持久化。

1.1.3 隐含功能

操作历史记录	针对 AVL 树等复杂结构，系统内部维护了操作历史栈，支持操作的撤销与重做，方便用户对比操作前后的树形结构变化。
随机数据生成	为 BST 构建、哈夫曼树输入等提供随机数据生成功能，便于快速测试和演示。
容错处理	对用户输入的非法数据（如非数字字符、格式错误的序列）进行捕获并弹窗或显示错误提示，防止程序崩溃。
视图自动缩放与适配	QGraphicsView 会根据当前画布内容的边界自动调整缩放比例和中心位置，确保所有节点都能完整显示在窗口内，避免图形溢出。

1.2 需要处理的数据

1.2.1 后端

后端主要负责数据的存储、逻辑运算和状态维护，不依赖具体的界面实现。

1.核心数据对象

数据类别	类名/类型	描述与属性
节点类	TreeNode	基础二叉树节点，包含 key (键值), left (左孩子指针), right (右孩子指针)。
	AVLNode	继承自 TreeNode，额外增加 height 属性用于计算平衡因子。
	HuffmanNode	包含 weight (权重), char (字符) 以及左右子节点指针，用于构建哈夫曼树。
	BTreeNode	包含 keys (关键字列表), children (子节点列表), leaf (是否叶子标记)。
	LinkedList._Node	链表节点，包含 value (数据域) 和 next (后继指针)。
容器类	Python list	用于模拟实现顺序表 (SequentialList) 和顺序栈 (SequentialStack) 的底层存储。
	Python dict	用于存储哈夫曼编码表（字符到二进制串的映射）以及图的邻接表信息（如果涉及）。

2.算法状态数据

生成器状态：使用 Python 生成器 (yield) 返回的字典对象来描述算法的中间状态。例如，背包问题算法产生的状态包含：stack (当前栈内容), remaining (剩余容量), action (当前操作类型 Push/Pop)。

序列化数据：用于文件存储的嵌套字典结构 (JSON 兼容格式)，例如 {'type': 'bst', 'tree': {'key': 10, 'left': ...}}。

1.2.2 前端

前端主要处理图形元素的属性数据，用于在屏幕上渲染模型。

图形项数据与交互

数据类别	具体内容	描述/用途
图形项数据	坐标数据	计算出的节点中心坐标 (x,y)，用于确定 QGraphicsEllipseItem (圆形节点) 或 QGraphicsRectItem (方形节点) 的位置。
	尺寸与样式	节点的宽高、填充颜色 (QBrush)、边框样式 (QPen)、文本字体 (QFont)。例如：查找时节点变黄，边框变红。
	连接关系	父子节点之间的连线 (QGraphicsLineItem) 起终点坐标，随节点位置动态更新。
交互数据	用户输入	用户在 QLineEdit 输入的字符串（如 "10,20,30" 或 "a:5, b:3"）。
	控制信号	用户点击按钮（如“下一步”、“插入”）或切换单选框产生的 Qt 信号 (Signal)，触发 Controller 执行逻辑。
	AI 对话流	AI 聊天记录窗口中的文本流，包含用户的自然语言指令和 AI 的反馈信息。

1.3 开发环境

操作系统：Windows 11
编程语言：Python 3.12
GUI 框架：PySide6
开发工具：Visual Studio Code

1.4 用户界面设计

系统采用经典的 左右侧边栏控制 + 主画布显示 的布局模式，界面简洁直观。
以下是主窗口布局的描述。

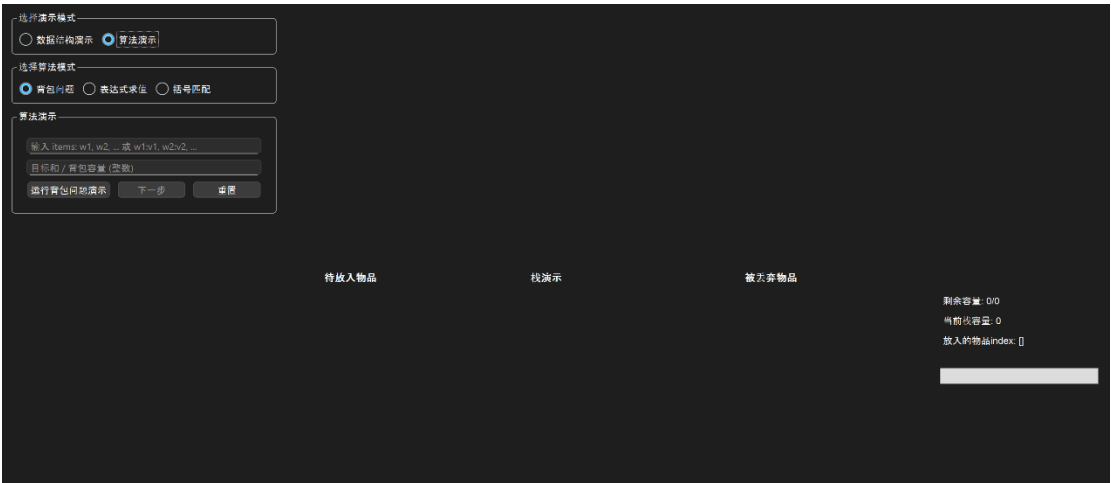
屏幕区域	模块名称	主要组件	功能描述
左侧面板	控制与导航	模式选择区 (QGroupBox)	包含单选按钮，用于在“数据结构演示”与“算法演示”两大核心模式间切换，触发布局刷新。
		结构/算法列表	列出支持的模型（如 BST, AVL, 背包问题等），用户点击切换当前活跃的后端模型。

		动态操作区	根据当前模式动态加载。包含通用输入框 (QLineEdit)、增删改查按钮、以及分步演示控制按钮（下一步/重置）。
		文件操作区	提供“保存结构”与“打开结构”按钮，支持 JSON 格式的数据持久化存取。
中央区域	可视化画布	QGraphicsView	基于 Qt 图形视图框架的核心画布。用于实时渲染节点、连线和动画。内置自动缩放算法(_auto_scale_view)，确保图形始终适配窗口大小。
		QGraphicsScene	管理所有图元（节点、线条、文本）的容器，支持全视口刷新 (FullViewportUpdate) 以保证动画流畅。
右侧面板	智能辅助	AI 助手对话框	固定宽度的侧边栏。包含只读的聊天记录显示框 (QTextEdit) 和指令输入框，提供自然语言交互入口。
		信息悬浮框	绘制在画布右上角的半透明文本框(QGraphicsRectItem)，显示当前算法步骤的详细文字说明（如“正在左旋...”、“栈顶元素为...”）。

下图是“数据结构演示”的 UI 设计



下图是“算法演示”的 UI 设计



2. 数据结构演示设计

本可视化模拟器基于 Python 3.12 开发，采用了面向对象编程的思想进行封装。为了实现数据逻辑与界面显示的解耦，项目整体遵循 Model-View-Controller 架构设计模式。所有核心数据结构类均定义在 `structures/` 目录下。

2.1 主要数据结构

本系统涵盖了线性结构与多种树形结构。由于 Python 语言的动态特性，基本存储结构主要依赖于 Python 内置的 `list` 以及自定义的节点类链接实现。

主要数据结构汇总如下表 2.1 所示：

数据结构类名	对应文件	逻辑结构	存储结构	说明
SequentialList	linear_list.py	线性结构	顺序存储	封装 Python list，模拟数组操作
LinkedList	linear_list.py	线性结构	链式存储	基于内部类 <code>_Node</code> 的单向链表
SequentialStack	stack.py	线性结构	顺序存储	基于 list 的后进先出（LIFO）结构
BinarySearchTree	binary_tree.py	树形结构	链式存储	基础二叉搜索树
AVLTree	avl_tree.py	树形结构	链式存储	增加高度属性的自平衡二叉树
HuffmanTree	huffman_tree.py	树形结构	链式存储	基于优先队列构建的最优二叉树
BTree	b_tree.py	树形结构	多路链式	多路平衡查找树

2.1.1 线性表与栈

1. 顺序表类 (SequentialList)

顺序表类 `SequentialList` 是对 Python 列表的封装，用于演示线性表的顺序存储特性。它主要通过操作内部的 `self._elements` 列表来实现增删改查。

其主要成员如下表 2.2 所示：

变量/函数	功能	实现逻辑
<code>self._elements</code>	存储数据的容器	初始化为空列表 <code>[]</code>

insert(index, value)	插入元素	检查索引越界后，调用 list 的 insert 方法
delete(index)	删除元素	检查索引有效性后，调用 list 的 pop 方法
get_all_elements()	获取所有数据	返回列表的浅拷贝，供视图层（View）绘制使用

2. 链表类 (LinkedList)

链表类采用嵌套类 `_Node` 来定义节点，通过指针（引用）连接。这种结构能够直观地演示节点在内存中的非连续存储及指针重连过程。

其主要成员如下表 2.3 所示：

变量/函数	功能	实现逻辑
class <code>_Node</code>	内部节点类	包含 <code>value</code> (数据域) 和 <code>next</code> (指针域)
<code>self._head</code>	头指针	指向链表的第一个节点，初始化为 <code>None</code>
insert(index, value)	插入节点	遍历找到插入位置的前驱，调整 <code>next</code> 指针指向
delete(index)	删除节点	遍历找到待删节点的前驱，断开链接

3. 栈类 (SequentialStack)

栈类主要用于演示“后进先出”的操作特性，这也是后续算法演示（如中缀转后缀、括号匹配）的核心基础结构。

其主要成员如下表 2.4 所示：

变量/函数	功能	实现逻辑
<code>self._elements</code>	栈容器	Python list，使用 <code>append/pop</code> 模拟栈顶操作
push(value)	入栈	<code>append(value)</code> ，元素添加至列表末尾（栈顶）
pop()	出栈	<code>pop()</code> ，移除并返回列表末尾元素
top()	查看栈顶	返回 <code>_elements[-1]</code> ，不删除元素

2.1.2 树形结构

1. 二叉树节点基类 (TreeNode) 与 AVL 节点 (AVLNode)

所有二叉树结构的基础是节点。对于普通 BST，使用 `TreeNode`；对于 AVL 树，继承并扩展了 `height` 属性以计算平衡因子。

树节点类如下表 2.5 所示：

类名	关键属性	说明
<code>TreeNode</code>	<code>key, left, right</code>	基础键值及左右子节点指针
<code>AVLNode</code>	<code>height</code>	在 <code>TreeNode</code> 基础上增加高度属性，初始化为 1
<code>HuffmanNode</code>	<code>weight, char</code>	增加权重和字符属性，重载了 <code><</code> 运算符用于堆排序

2. AVL 树类 (AVLTree)

`AVLTree` 是本系统的核心难点之一，它在 `BinarySearchTree` 的基础上增加了旋转平衡逻辑。

`AVLTree` 类如下表 2.6 所示：

变量/函数	功能	实现逻辑
-------	----	------

self.root	根节点	树的入口
_get_balance(node)	计算平衡因子	左子树高度 - 右子树高度
_right_rotate(y)	右旋转	处理 LL 型失衡，更新涉及节点的高度
_left_rotate(x)	左旋转	处理 RR 型失衡，更新涉及节点的高度
insert(key)	插入节点	递归插入，回溯时更新高度并检测平衡因子，必要时旋转

3. B-树类 (BTree)

B-树用于演示多路查找树的分裂过程，采用了非递归与递归结合的方式实现。

B-Tree 类如下表 2.7 所示：

变量/函数	功能	实现逻辑
self.t	最小度数	决定节点关键字数量范围 $[t - 1, 2t - 1]$
BTreeNode	节点类	包含 keys (列表) 和 children (列表)
split_child(parent, i)	节点分裂	将满节点分裂为两个，中间关键字上移至父节点
insert(key)	插入关键字	检查根节点是否满，满则分裂根，否则调用辅助插入函数

2.2 程序架构

2.2.1 整体架构

本程序采用了经典的 **Model-View-Controller** 架构设计，将数据存储、逻辑控制与界面渲染完全分离，保证了代码的可维护性与扩展性。

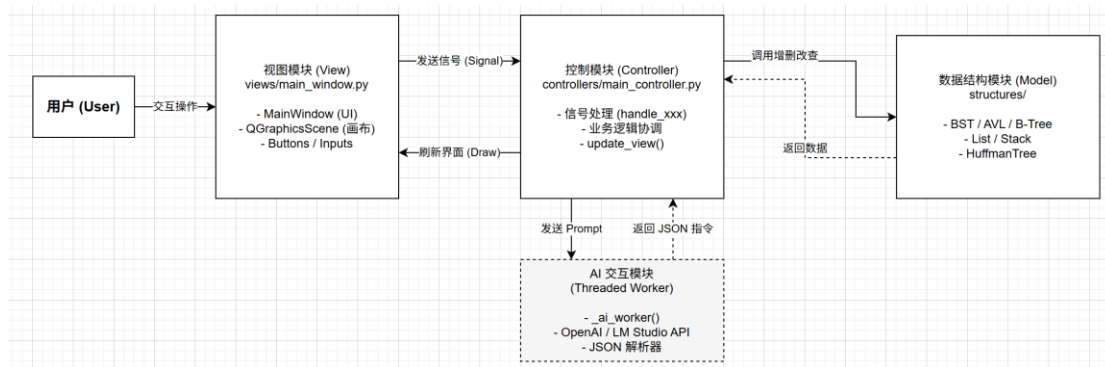
整体文件结构概览如下表 2.8 所示：

程序架构	文件夹/文件	功能描述	功能模块划分
Model (模型层)	structures/	包含所有数据结构的类定义及核心逻辑算法	数据结构模块
View (视图层)	views/main_window.py	基于 PySide6 构建主窗口，负责图形绘制 (QGraphicsScene)	UI 界面模块
Controller (控制层)	controllers/main_controller.py	处理用户交互信号，协调 Model 数据更新并通知 View 重绘	逻辑控制模块
Entry (入口)	main.py	程序启动入口，初始化 Application	启动模块

2.2.2 核心模块关系

如下图 2.1 所示 (架构示意图)，用户在 MainWindow (View) 上的操作 (如点击“插入”按钮) 会发送信号给 MainController。Controller 调用 structures (Model) 中的具体方法修改数据状态，随后 Controller 获取最新的数据快照，并调用 View 中的绘图方法 (如 draw_bst) 在画布上重新渲染。

这种设计使得我在添加新的数据结构时，只需在 structures/ 下新增类文件，并在 Controller 中注册即可，无需大幅修改界面代码。



2.2.3 线性结构模块

该模块负责顺序表、链表和栈的逻辑维护与可视化渲染。它不仅实现了基本的数据存储，还专门为“插入”和“删除”操作设计了分步可视化逻辑，以展示内存变动或指针修改的过程。

该模块构成见下表 2.9:

细分模块	核心函数/类	功能	实现
顺序表	SequentialList.insert	数组元素移动与插入	基于 Python list 的 insert，底层自动处理容量。
	gen_seq_ins (Controller)	生成插入动画帧	使用生成器 yield 返回中间状态（数组元素逐个右移），视图层据此高亮移动过程。
链表	LinkedList._Node	定义链式节点	嵌套类封装 value 和 next 指针。
	draw_linked_list (View)	绘制节点与指针	遍历链表，计算节点坐标，并使用 QGraphicsLineItem 和三角箭头绘制逻辑指针指向。
	gen_linked_ins (Controller)	分步演示指针重连	将插入过程分为 place(放置新节点) 和 relink(修改 next 指针) 两个阶段进行展示。
栈	SequentialStack	限制访问端的线性表	封装 list，仅暴露 push/pop 接口，严格遵循 LIFO 原则。
	draw_stack (View)	栈的可视化	采用“基准线+向上堆叠”的绘制策略，新元素入栈时会有颜色高亮提示。

上述函数在模块内被 UI 信号触发调用。为了实现动态效果，Controller 中的操作函数（如 handle_seq_insert_start）并不直接修改数据，而是初始化一个 Python 生成器 (Generator)。用户点击“下一步”时，Controller 从生成器获取当前的数组快照或指针状态，并通知 View 进行重绘。这种机制成功地将瞬时的内存操作转化为可被人眼捕捉的连续动画步骤。

2.2.4 二叉树模块

该模块涵盖了二叉搜索树 (BST)、AVL 树以及通用二叉树的构建与操作。核心难点在于如何动态计算树的节点坐标以保证布局美观，以及如何可视化递归操作的路径。

该模块构成见下表 2.10:

细分模块	核心函数/类	功能	实现
基础 BST	BinarySearchTree	递归插入与删除	采用标准递归逻辑维护 BST 性质（左小右大）。
	search_path (Generator)	可视化查找路径	在递归查找过程中 yield 当前访问的节点，视图层据此将路径上的节点标红高亮。
	_get_node_positions (View)	自动布局算法	采用“中序遍历确定 X 坐标，前序遍历确定 Y 坐标”的经典算法，确保节点互不遮挡且结构清晰。
AVL 树	AVLTree._left/right_rotate	树的旋转与平衡	实现了 LL、RR、LR、RL 四种旋转逻辑，并在旋转后自动更新节点高度。
	_avl_history (Controller)	操作回退 (Undo)	维护一个栈来存储每次操作后的树结构深拷贝快照，实现了“上一步/下一步”的历史回溯功能。
通用树	build_from_pre_in	从遍历序列还原树	解析前序+中序序列，通过递归逻辑重建二叉树结构，并分步演示重建过程。

本模块在实现 AVL 树时，为了让用户理解自平衡过程，特别在 insert 方法中记录了 rotation info（旋转类型与涉及节点）。当发生失衡时，视图层不仅会重绘树结构，还会读取该信息并在画布旁通过文字提示“检测到 LL 型失衡，正在进行右旋...”，极大地增强了教学效果。

2.2.5 高级树形结构模块

该模块展示了哈夫曼树和 B-树 两种更复杂的数据结构，重点在于展示它们特殊的构建规则和节点分裂逻辑。

该模块构成见下表 2.11:

细分模块	核心函数/类	功能	实现
哈夫曼树	HuffmanTree.build_step_by_step	分步构建最优二叉树	使用 Python 的 heapq 模块维护优先队列。生成器逐步 yield 队列状态，展示“取出最小两个节点 -> 合并 -> 放回”的完整过程。
	draw_huffman_tree	绘制编码树	递归绘制树结构，并在视图右侧生成并显示每个字符的哈夫曼编码表。
B-树	BTree.split_child	节点分裂逻辑	当节点关键字数量达到 $2t-1$ 时，将其分裂为两个子节点，并将中间关键字提升至父节点。
	_measure_btree (View)	多路树布局计算	递归计算每个子树所需的总宽度，动态调整父节点的横向位置，确保多路子节点排列整齐且不重叠。

哈夫曼树的构建过程采用了完全的“帧动画”模式：Controller 维护一个

`_current_huffman_build_state` 变量，每次点击“下一步”，生成器返回新的队列状态（包含离散的节点森林或局部子树），视图层根据这些离散元素实时计算布局并绘制，生动还原了“森林变树”的过程。

2.2.6 AI 智能交互模块

该模块是本系统的创新点，利用大语言模型 (LLM) 实现了自然语言到代码指令的转换，使用户可以通过对话控制数据结构。我使用的本地 iGPU 运行的模型（LM Studio 提供的本地 AI 服务器功能），模型为 Gemma 3n E4b，为 6.9B 参数量的无思考模型。

该模块构成见下表 2.12:

细分模块	核心函数	功能	实现
指令解析	<code>handle_ai_request</code>	发送用户指令	获取用户输入的自然语言（如“删掉 5”），构建包含当前上下文的 System Prompt，发送给 LLM。
异步通信	<code>_ai_worker (Thread)</code>	后台请求 LLM	为了防止网络请求阻塞 UI 主线程，采用 Python threading 开启独立线程进行 API 调用。
指令执行	<code>_process_ai_response</code>	解析并执行 JSON	接收 LLM 返回的 JSON 指令（如 <code>{"action": "delete", "value": 5}</code> ），映射到 Controller 的具体操作函数。

该模块的设计遵循了“Human-in-the-loop”原则。AI 并不直接修改内存数据，而是将用户的自然语言翻译成标准的 JSON 中间指令。Controller 接收到指令后，复用已有的 `handle_insert` 或 `handle_delete` 等逻辑进行执行。这种设计既利用了 AI 的理解能力，又保证了底层数据操作的严谨性和安全性。

3. 算法演示设计

本系统在实现基础数据结构可视化的基础上，进一步集成了经典算法的动态演示模块。与数据结构演示类似，算法演示同样遵循 MVC 架构，但其核心在于利用 Python 的生成器机制，将连续的算法执行过程离散化为可控的“帧”序列，从而实现“单步调试”级的可视化效果。

3.1 主要算法

本系统实现的算法主要围绕“栈”的应用以及经典的“回溯法”展开。所有核心算法代码均位于 `algorithms/` 目录下。

算法模块汇总如下表 3.1 所示:

算法名称	对应文件	核心思想	可视化机制
0/1 背包问题	<code>knapsac.py</code>	基于栈的非递归回溯 (DFS)	记录入栈/出栈操作、剩余容量及被丢弃物品，每步 Yield 状态
中缀转后缀	<code>infix_to_postfix.py</code>	调度场算法、栈的应用	利用栈处理运算符优先级，Yield 栈与后缀表达式的变化
后缀表达式求值	<code>postfix_evaluation.py</code>	栈式计算	扫描操作数入栈，遇运算符弹出计算后回栈，Yield 计算过程
括号匹配	<code>parentheses_matching.</code>	栈式匹配	左括号入栈，右括号匹配出栈，Yield

	py		栈状态及匹配结果
--	----	--	----------

3.1.1 背包问题（基于栈的回溯）

该模块演示了如何解决经典的子集和问题，即在给定的物品集合中寻找总重等于目标容量的组合。

为了清晰地展示深度优先搜索（DFS）的回溯过程，代码中放弃了传统的系统递归栈，转而使用自定义的 `SequentialStack` 来显式管理搜索路径。

算法设计如下表 3.2 所示：

项目	描述 / 实现逻辑
函数入口	<code>solve_subset_sum_backtracking_gen(weights, capacity)</code>
输入数据	<code>weights</code> : 物品重量列表; <code>capacity</code> : 背包目标容量
核心数据结构	<code>stack</code> : <code>SequentialStack</code> ，存储当前路径选中的物品索引
关键状态 Yield	- <code>push</code> : 尝试放入物品，栈增加，容量减少 - <code>pop</code> : 回溯（剪枝或无解），栈减少，容量恢复 - <code>found</code> : 找到解，返回完整路径
剪枝策略	1. 若 <code>current_rem < 0</code> ，说明容量透支，触发回溯 2. 若 <code>current_rem - w < 0</code> ，预判放入即超重，跳过当前物品

3.1.2 栈的应用算法

为了展示栈在编译器设计和文本处理中的重要作用，系统实现了三个相关算法。

几种栈应用算法如下表 3.3 所示：

算法	核心逻辑描述	状态快照内容
中缀转后缀	遍历表达式： 1. 操作数直接输出。 2. 左括号入栈。 3. 右括号则弹出栈顶直到遇到左括号。 4. 运算符则根据优先级弹出栈顶元素。	- <code>token</code> : 当前扫描字符 - <code>stack</code> : 运算符栈当前内容 - <code>postfix</code> : 当前生成的后缀串 - <code>action_text</code> : 操作说明文字
后缀求值	遍历后缀表达式： 1. 数字入栈。 2. 遇运算符，弹出栈顶两个元素计算，结果回栈。	- <code>stack</code> : 操作数栈 - <code>calc_info</code> : { <code>v1</code> , <code>v2</code> , <code>op</code> }，用于动画展示计算细节
括号匹配	1. 左括号 (<code>{</code>) 入栈。 2. 右括号 (<code>}</code>) 与栈顶匹配。 3. 结束后栈为空则匹配成功。	- <code>stack</code> : 符号栈 - <code>step_type</code> : <code>match</code> (成功) / <code>error</code> (失配)

3.2 程序架构

3.2.1 整体架构

算法演示部分的整体架构与第 2 章所述的 MVC 架构保持一致（Model-View-Controller 分离）。

Model (模型): `algorithms/` 文件夹下的纯算法实现，不包含任何 GUI 代码，仅负责逻辑

运算和状态生成。

View (视图): `views/main_window.py` 中对应的 `draw_xxx_demo` 方法, 负责根据 Model 传来的数据重绘界面。

Controller (控制): `controllers/main_controller.py`, 负责协调用户点击与算法生成器的步进。

关于 MVC 的通用架构原理已在 2.2 节中详细阐述, 此处不再赘述, 仅针对算法演示模块的特殊实现——生成器驱动机制进行说明。

3.2.2 核心模块关系

不同于数据结构演示中“用户操作 -> 修改数据 -> 立即重绘”的模式, 算法演示是一个连续的时间过程。为了在 GUI 中不阻塞主线程并实现“上一步/下一步”控制, 本系统在架构上采用了 Python 的 `yield` 关键字构建状态机。

各层级在算法演示中的具体职责如下表 3.4 所示:

架构层级	模块/类	职责描述	数据流向
Controller	MainController	状态管理者。 1. 维护 <code>_knapsack_gen</code> , <code>_infix_gen</code> 等生成器实例。 2. 响应“下一步”按钮, 调用 <code>next(generator)</code> 。 3. 解析 <code>yield</code> 返回的字典, 提取 UI 所需参数。	Input -> Generator -> State Dict
Model	algorithms/*.py	状态生产者。 1. 执行算法逻辑。 2. 在关键步骤（如入栈、比较、计算）暂停执行 (<code>yield</code>)。 3. 返回当前时刻的内存快照（如栈的拷贝、当前索引）。	Yield State -> Controller
View	MainWindow	状态渲染者。 1. <code>draw_knapsack_backtracking_demo</code> : 绘制物品列、栈区、丢弃区。 2. <code>draw_infix_postfix_demo</code> : 绘制字符流、符号栈、结果串。 3. 接收 Controller 传来的高亮参数（如 <code>highlighted_action</code> ），用颜色标记当前操作对象。	State Dict -> Graphics Items

这种架构设计的优势在于: 算法逻辑与 UI 渲染完全解耦。算法工程师只需关注在代码的关键位置插入 `yield` 语句返回当前关注的变量, 而无需编写任何界面更新代码; 界面开发人员只需根据收到的状态字典绘制静态帧, 无需理解复杂的递归或回溯逻辑。

4. 详细设计

本章将对系统中的核心算法逻辑进行详细说明, 重点介绍 AVL 树的平衡维护、B-树的节点分裂、可视化系统的生成器驱动机制以及 AI 智能交互的实现流程。

4.1 数据结构核心算法

4.1.1 AVL 树插入与自平衡 (`AVLTree.insert`)

AVL 树作为自平衡二叉搜索树，其核心难点在于插入节点后如何通过旋转操作维持树的平衡。本系统在 `structures/avl_tree.py` 中实现了基于递归的插入与平衡调整。

1. 算法说明

函数 `_insert_recursive` 是实现该功能的关键。其逻辑如下：

标准 BST 插入：首先按照二叉搜索树规则递归找到插入位置并创建新节点。

更新高度：在递归回溯过程中，更新当前节点的高度属性。

计算平衡因子：计算当前节点的平衡因子（左子树高度 - 右子树高度）。

旋转调整：若平衡因子绝对值大于 1，则根据新节点插入位置与当前节点的关系，判定属于 LL、RR、LR 或 RL 四种情况之一，并调用相应的 `_left_rotate` 或 `_right_rotate` 函数进行旋转。

信息记录：为了配合前端可视化，算法在执行旋转前会将旋转类型（如 "LL"）和涉及的节点记录在 `rotation_info` 字典中，供视图层展示“检测到失衡，正在右旋”等提示。

2. 核心代码

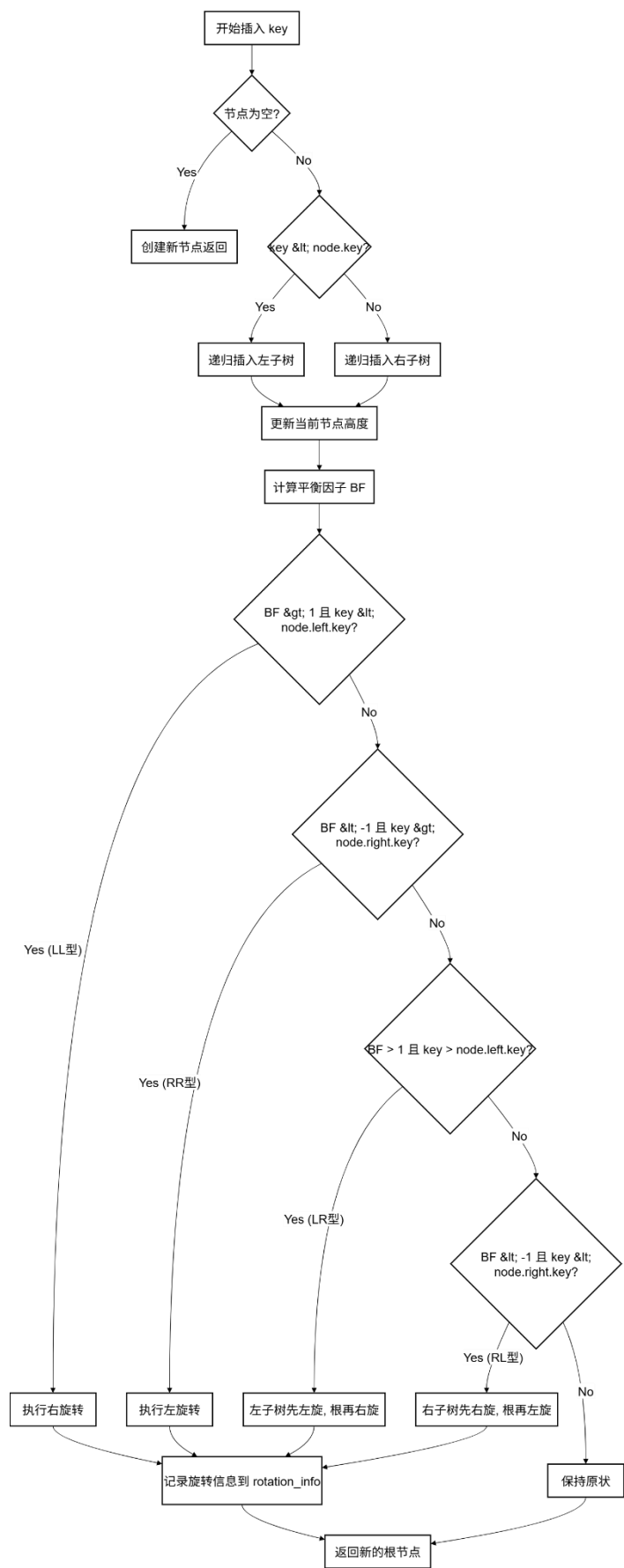
```
1. def _insert_recursive(self, node, key):
2.     # 执行标准的 BST 插入
3.     if not node:
4.         return AVLNode(key)
5.     elif key < node.key:
6.         node.left = self._insert_recursive(node.left, key)
7.     elif key > node.key:
8.         node.right = self._insert_recursive(node.right, key)
9.     else:
10.        return node # 不允许重复键
11.
12.    # 更新祖先节点的高度
13.    node.height = 1 + max(self._get_height(node.left), self._get_height(node.right))
14.
15.    # 获取平衡因子并检查是否需要旋转
16.    balance = self._get_balance(node)
17.
18.    # 根据四种情况执行旋转
19.    # LL Case
20.    if balance > 1 and key < node.left.key:
21.        self.rotation_info = {
22.            "type": "LL",
23.            "node": node,
24.            "balance": balance,
25.            "message": f"节点{node.key}的平衡因子为{balance} > 1, 且新节点{key}插入在左子树的左边, 执行右旋转"
26.        }
27.        return self._right_rotate(node)
28.
29.    # RR Case
30.    if balance < -1 and key > node.right.key:
```

```

31.         self.rotation_info = {
32.             "type": "RR",
33.             "node": node,
34.             "balance": balance,
35.             "message": f"节点{node.key}的平衡因子为{balance} < -1, 且新节点{key}插入在右
子树的右边, 执行左旋转"
36.         }
37.         return self._left_rotate(node)
38.
39.     # LR Case, 先预选转变成 LL, 再执行右旋
40.     if balance > 1 and key > node.left.key:
41.         self.rotation_info = {
42.             "type": "LR",
43.             "node": node,
44.             "balance": balance,
45.             "message": f"节点{node.key}的平衡因子为{balance} > 1, 且新节点{key}插入在左
子树的右边, 先对左子树执行左旋转, 再对根执行右旋转"
46.         }
47.         node.left = self._left_rotate(node.left)
48.         return self._right_rotate(node)
49.
50.     # RL Case, 先变成 RR, 再执行左旋
51.     if balance < -1 and key < node.right.key:
52.         self.rotation_info = {
53.             "type": "RL",
54.             "node": node,
55.             "balance": balance,
56.             "message": f"节点{node.key}的平衡因子为{balance} < -1, 且新节点{key}插入在右
子树的左边, 先对右子树执行右旋转, 再对根执行左旋转"
57.         }
58.         node.right = self._right_rotate(node.right)
59.         return self._left_rotate(node)
60.
61.     return node

```

3. 算法流程图



4.1.2 B-树节点分裂 (BTree.split_child)

B-树作为多路平衡查找树，其核心特性在于通过节点分裂来处理上溢。本系统在 `structures/b_tree.py` 中实现了节点分裂逻辑。

1. 算法说明

当插入操作导致某节点的关键字数量达到上限 ($2t - 1$) 时，`split_child` 函数会被调用。该函数的作用是将一个满子节点分裂为两个，并将中间的关键字提升到父节点中。

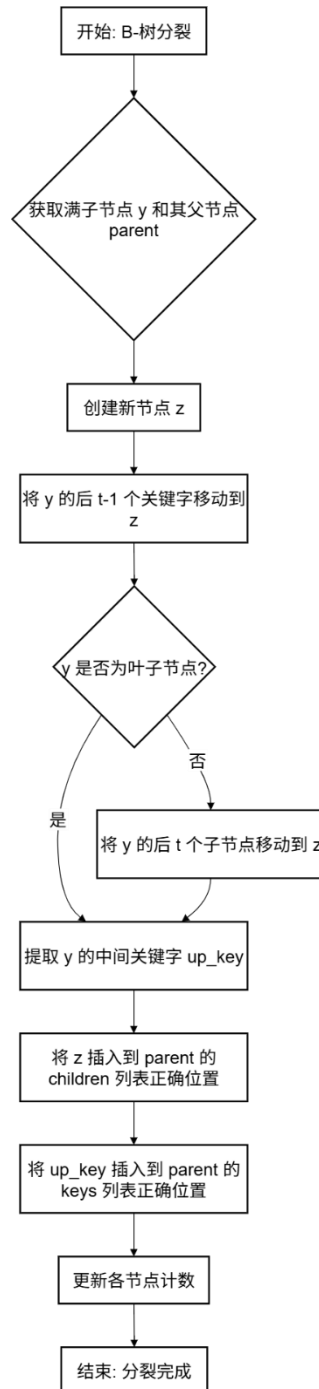
算法步骤如下：

1. **创建新节点：**创建一个新的节点 `z`，用于存储原满节点 `y` 后半部分的关键字。
2. **分配关键字：**将 `y` 中索引为 t 到 $2t - 2$ 的关键字移动到 `z` 中。如果 `y` 不是叶子节点，还需将其后半部分子节点移动到 `z`。
3. **调整父节点：**将新节点 `z` 插入到父节点 `parent` 的子节点列表中，同时将 `y` 的中间关键字（索引为 $t - 1$ ）提升插入到 `parent` 的关键字列表中。
4. **保存结果：**更新各节点属性，完成分裂。

2. 核心代码

```
1. def split_child(self, parent, index):
2.     """
3.     Split the full child of parent at index.
4.     """
5.     t = self.t
6.     y = parent.children[index]
7.     z = BTreeNode(t, leaf=y.leaf)
8.
9.     # Middle key to move up
10.    up_key = y.keys[t - 1]
11.
12.    # z gets the last t-1 keys of y
13.    z.keys = y.keys[t:]
14.    # y retains the first t-1 keys
15.    y.keys = y.keys[:t - 1]
16.
17.    # If y is not leaf, split its children
18.    if not y.leaf:
19.        z.children = y.children[t:]
20.        y.children = y.children[:t]
21.
22.    # Insert new child z into parent
23.    parent.children.insert(index + 1, z)
24.    parent.keys.insert(index, up_key)
```

3. 算法流程图



4.2 可视化系统逻辑设计

4.2.1 基于生成器的分步演示机制

为了满足课程设计中“动态过程”和“分步演示”的要求，本系统并未采用传统的定时器循环，而是创新性地使用了 Python 的生成器与 yield 关键字。本节以**顺序表分步删除**为例进行说明。

1. 算法说明

在 controllers/main_controller.py 中，删除操作并非瞬间完成，而是被封装在一个生成器函数 gen_del 中：

状态快照：在算法执行的关键步骤（如查找目标、元素移动、清空尾部），使用 `yield` 暂停执行，并返回一个包含当前数据快照（`display_array`）和操作描述的字典。

UI 响应：`MainController` 的 `handle_seq_delete_next` 函数负责调用 `next(iterator)`。每当用户点击“下一步”，系统从上次暂停处继续执行，直到遇到下一个 `yield`。

视觉同步：控制器接收到 `yield` 返回的数据后，调用 `update_view`，视图层根据数据中的 `step` 类型（如 `"shift"` 或 `"clear_tail"`）绘制高亮框或动画效果。

这种设计将复杂的时序控制逻辑从 UI 线程中剥离，避免了界面卡死，同时实现了精确的单步控制。

2. 核心代码

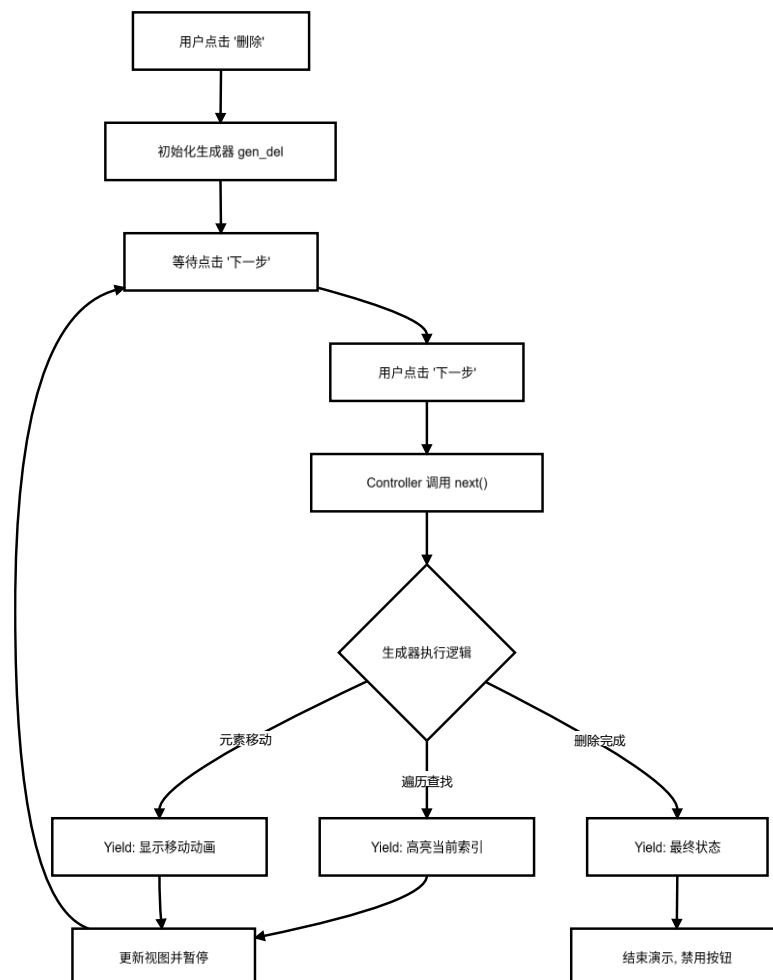
```
1. def gen_del():
2.     elems = self._models['sequential_list'].get_all_elements()
3.     n = len(elems)
4.     # 找到第一个匹配的索引
5.     target_idx = None
6.     for i, v in enumerate(elems):
7.         if v == value:
8.             target_idx = i
9.             break
10.    yield {"index": i, "deleted": False}
11.
12.    if target_idx is None:
13.        # 未找到，结束
14.        return
15.
16.    # 模拟移动：对于 j 从 target_idx+1 到 n-1，依次将 elems[j] 移到位置 j-1
17.    working = elems.copy()
18.    for j in range(target_idx + 1, n):
19.        working[j-1] = working[j]
20.        display = working.copy()
21.        # 将当前中间状态传给视图
22.        yield {"display_array": display, "index": j-1, "deleted": False, "step":
"shift", "from": j, "to": j-1}
23.
24.    # 现在最后一位应显示为空
25.    display = working.copy()
26.    display[-1] = None
27.    yield {"display_array": display, "index": n-1, "deleted": False, "step":
"clear_tail"}
28.
29.    # 执行模型删除（真正缩减长度）
30.    try:
31.        self._models['sequential_list'].delete(target_idx)
32.    except Exception:
```

```

33.         pass
34.
35.         # 最终返回已删除的信号，视图将以模型为准渲染最终数组（长度已减一）
36.         yield {"index": target_idx, "deleted": True}

```

3. 交互流程图



4. 3 扩展功能设计

4. 3. 1 AI 智能指令解析

系统集成了大语言模型接口，实现了自然语言到系统指令的映射。该功能位于 `controllers/main_controller.py`。

1. 算法说明

该模块，AI 不直接操作内存，而是充当翻译器：

构建 Prompt: 将用户的自然语言（如“帮我删掉 5”）与当前系统状态、允许的 JSON 指令集模板拼装成 System Prompt。

异步请求: 通过 `_ai_worker` 线程向本地 LLM 服务（如 LM Studio）发送请求，避免阻塞 UI。

指令解析: 在 `_process_ai_response` 中解析返回的 JSON 数据。系统设计了容错机制，既支持标准 JSON 解析，也支持正则提取。

动作分发: 根据解析出的 action（如 `insert`, `delete`, `switch_structure`），反射调用控制器中

已有的处理函数（如 `handle_insert`），实现了 AI 逻辑与业务逻辑的复用。

2. 核心代码

```
1. try:
2.     for act in actions:
3.         action_type = act.get("action")
4.
5.         if action_type == "switch_structure":
6.             struct = act.get("structure")
7.             self._view.ai_chat_display.append(f"<i>AI: 切换到 {struct}...</i>")
8.             if hasattr(self._view, f"radio_{struct}"):
9.                 getattr(self._view, f"radio_{struct}").setChecked(True)
10.                 from PySide6.QtCore import QApplication
11.                 QApplication.processEvents()
12.
13.         elif action_type == "insert":
14.             values = act.get("values", [])
15.             if isinstance(values, int) or isinstance(values, str):
16.                 values = [values]
17.
18.             self._view.ai_chat_display.append(f"<i>AI: 插入 {values}...</i>")
19.
20.             # 如果是哈夫曼树，批量处理所有插入值，一次性更新
21.             if self._active_model_name == "huffman_tree":
22.                 try:
23.                     current_text = self._view.huffman_input.text().strip()
24.                     items = []
25.                     if current_text:
26.                         items.append(current_text)
27.
28.                     for val in values:
29.                         val_str = str(val)
30.                         # 格式化处理
31.                         if ":" not in val_str:
32.                             new_item = f"{val_str}:{val_str}"
33.                         else:
34.                             new_item = val_str
35.                         items.append(new_item)
36.
37.                     # 用逗号连接
38.                     new_full_text = ", ".join(items)
39.                     self._view.huffman_input.setText(new_full_text)
40.
41.                     # 触发绘制
42.                     self.handle_huffman_draw()
```



```

43.
44.         self._view.display_info_text(f"已批量添加哈夫曼节点: {values}")
45.     except Exception as e:
46.         print(f"哈夫曼插入失败: {e}")
47.
48.     else:
49.         # 其他结构 (BST, 顺序表等) 逐个处理
50.         for val in values:
51.             try:
52.                 val_int = int(val)
53.
54.                 if self._active_model_name in ["sequential_list",
"linked_list"]:
55.                     model = self._models[self._active_model_name]
56.                     model.insert(model.get_length(), val_int)
57.                     self.update_view()
58.                 else:
59.                     self._view.value_input.setText(str(val_int))
60.                     self.handle_insert()
61.
62.                 from PySide6.QtCore import QCoreApplication
63.                 QCoreApplication.processEvents()
64.             except Exception as e:
65.                 print(f"插入值 {val} 失败: {e}")
66.
67.         # 处理 Pop 指令
68.     elif action_type == "pop":
69.         try:
70.             self._view.ai_chat_display.append(f"<i>AI: 执行出栈...</i>")
71.         except Exception:
72.             pass
73.         try:
74.             # 使用专门的栈弹出处理以获得更好的可视化与提示
75.             self.handle_stack_pop()
76.         except Exception:
77.             pass
78.
79.     elif action_type == "delete":
80.         val = act.get("value")
81.         del_type = act.get("type")
82.
83.         # 按索引删除
84.         if del_type == "index":
85.             try:

```

```

86.         idx = int(val)
87.         try:
88.             self._view.ai_chat_display.append(f"<i>AI: 删除索引
{idx}...</i>")
89.         except Exception:
90.             pass
91.         try:
92.             self.handle_delete_by_index(idx)
93.         except Exception:
94.             pass
95.     except Exception:
96.         # 无效索引值, 回退为按值删除
97.         try:
98.             self._view.ai_chat_display.append(f"<i>AI: 删除 {val}...</i>")
99.         except Exception:
100.            pass
101.        try:
102.            self._view.value_input.setText(str(val))
103.            self.handle_delete()
104.        except Exception:
105.            pass
106.
107.        # 默认按值删除
108.    else:
109.        try:
110.            self._view.ai_chat_display.append(f"<i>AI: 删除值 {val}...</i>")
111.        except Exception:
112.            pass
113.        try:
114.            self._view.value_input.setText(str(val))
115.            self.handle_delete()
116.        except Exception:
117.            pass
118.
119.    elif action_type == "search":
120.        val = act.get("value")
121.        # 如果模型仍发送旧的 "search" 指令来查询栈顶, 兼容为 get top
122.        try:
123.            sval = str(val).strip().lower()
124.        except Exception:
125.            sval = None
126.
127.        if sval in ("top", "stack top") and self._active_model_name == "stack":
128.            try:

```

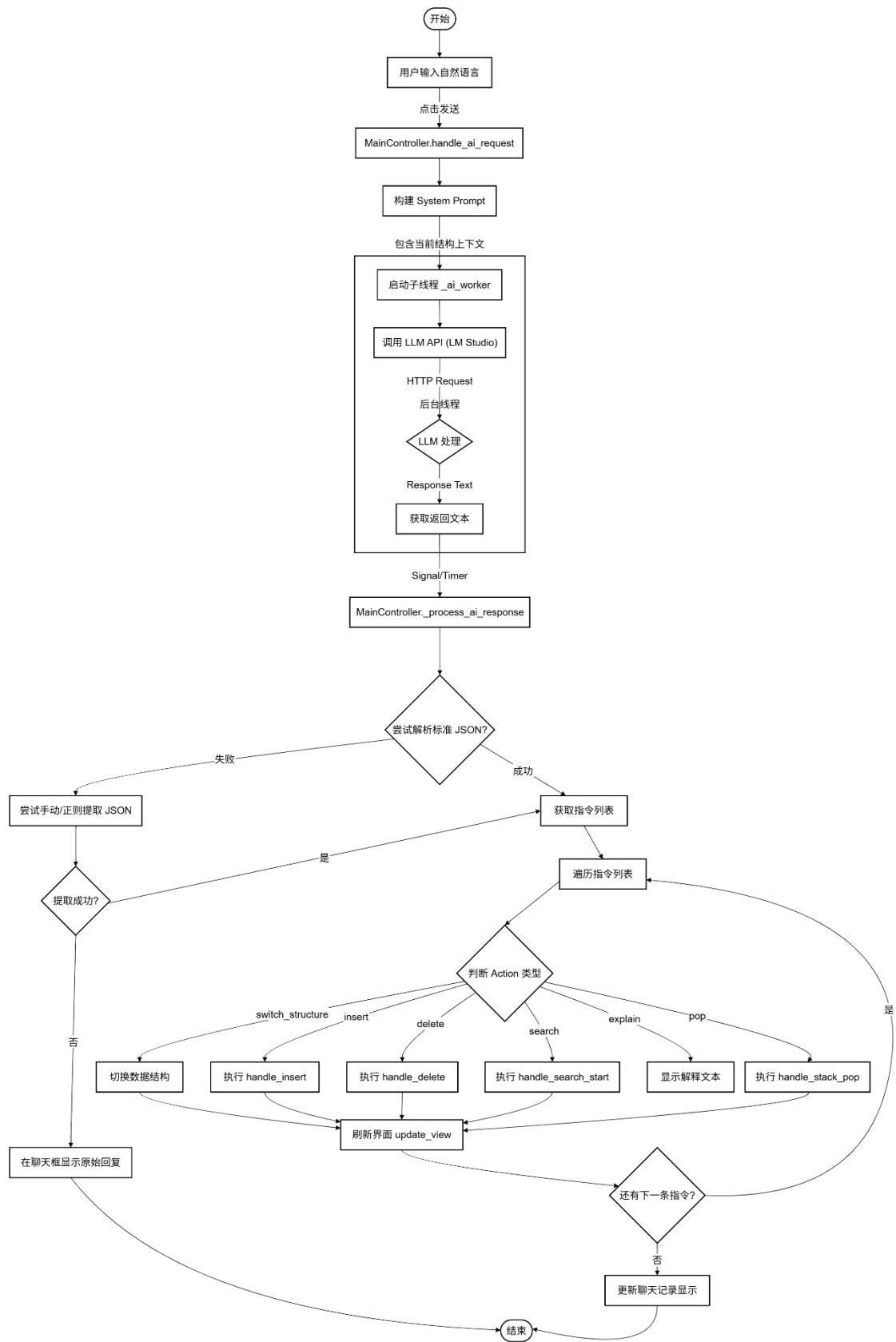
```

129.         self._view.ai_chat_display.append(f"<i>AI: 查看栈顶 (兼容)...</i>")
130.     except Exception:
131.         pass
132.     try:
133.         self.handle_stack_top()
134.     except Exception:
135.         pass
136. else:
137.     try:
138.         self._view.ai_chat_display.append(f"<i>AI: 查找 {val}...</i>")
139.     except Exception:
140.         pass
141.     try:
142.         self._view.value_input.setText(str(val))
143.         self.handle_search_start()
144.     except Exception:
145.         pass
146.
147.     # 处理 Get/Peek 指令
148.     elif action_type == "get":
149.         get_type = act.get("type")
150.
151.         # 1. 查看栈顶 - 使用专门的 get/top 指令，不要使用 search
152.         if get_type == "top":
153.             try:
154.                 self._view.ai_chat_display.append(f"<i>AI: 查看栈顶...</i>")
155.             except Exception:
156.                 pass
157.             try:
158.                 # 复用已有的栈顶查看实现（与 UI 按钮行为一致）
159.                 self.handle_stack_top()
160.             except Exception:
161.                 pass
162.
163.         # 2. 查看特定索引（用于顺序表/链表）
164.         elif get_type == "index":
165.             val = act.get("value")
166.             try:
167.                 idx = int(val)
168.                 try:
169.                     self._view.ai_chat_display.append(f"<i>AI: 查看索引
{idx}...</i>")
170.                 except Exception:
171.                     pass

```

```
172.         try:
173.             self.handle_get_by_index(idx)
174.         except Exception:
175.             pass
176.     except Exception:
177.         # 无效索引值，忽略该指令
178.         pass
179.
180.     elif action_type == "explain":
181.         self._view.ai_chat_display.append(f"<b>AI:</b> {act.get('text')}")
182.
183. except Exception as e:
184.     print(f"处理指令出错: {e}")
185.     self._view.ai_chat_display.append(f"<b>AI Error:</b> 无法解析指令")
```

3. 交互流程图



5. 测试

为了验证“数据结构可视化模拟器”的功能完整性与稳定性，本章设计了覆盖核心功能、异常处理及边界条件的系统测试方案。测试方法采用黑盒测试，通过 GUI 界面进行交互，观察可视化结果与状态反馈是否符合预期。

5.1 正常运行测试

5.1.1 线性结构操作测试

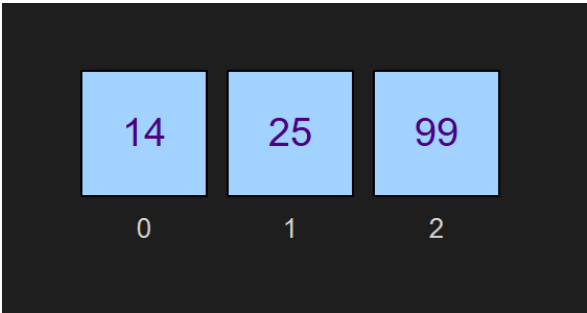
测试目标：验证顺序表、链表及栈的基本增删查改与动画反馈。

测试用例 1：顺序表插入与移动

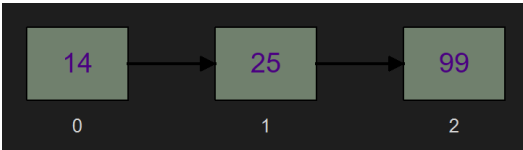
操作：选择“顺序表”模式，在索引 1 后插入数值 99（当前列表已有元素 14、25）。

预期结果：系统应高亮索引 1 位置，最后将 99 填入空位。

测试结果：如图 5.1 所示，最终数据插入正确。



同理，链表结果如下：



测试用例 2：栈的入栈与出栈

- 操作：**选择“栈”模式，依次输入 10, 20, 30 点击“入栈”，随后点击一次“出栈”。
- 预期结果：**元素 10, 20, 30 依次从下至上堆叠显示，新入栈元素有颜色高亮。点击出栈后，栈顶 30 被移除，并在右侧短暂显示被弹出的元素值，提示区显示“已出栈：30”。随后查看栈顶，应当显示 “20”
- 测试结果：**如图 5.2 所示，栈的 LIFO（后进先出）特性、查看栈顶展示正确。



5.1.2 树形结构构建与操作测试

测试目标：验证 BST 的动态构建、查找路径高亮及删除逻辑。

测试用例 3：二叉搜索树 (BST) 随机构建与查找

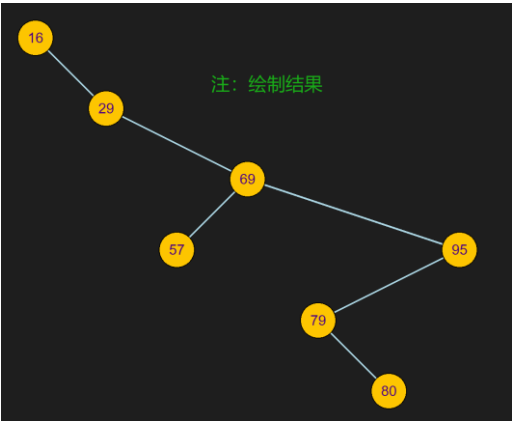
操作：选择“二叉搜索树”模式，点击“随机生成序列”，随后点击“构建 BST”。构建

完成后，在通用输入框输入一个存在的节点值（例如根节点的值），点击“开始查找”。

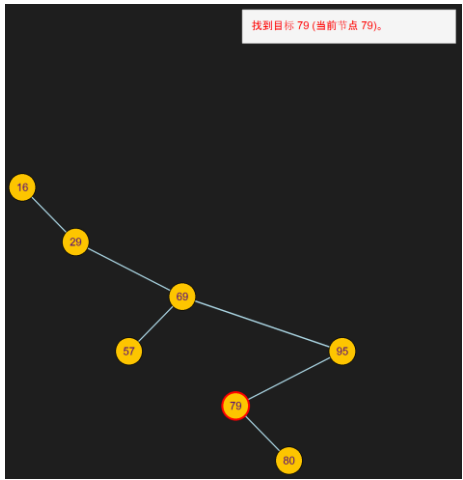
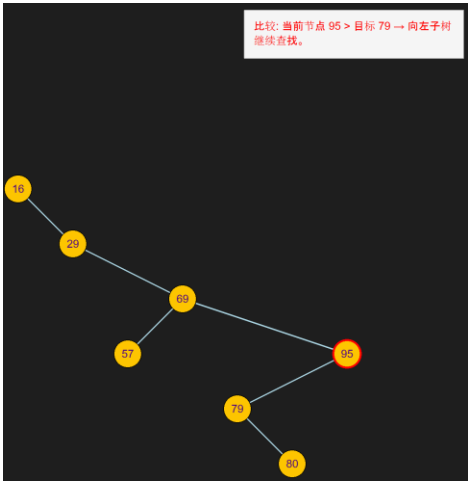
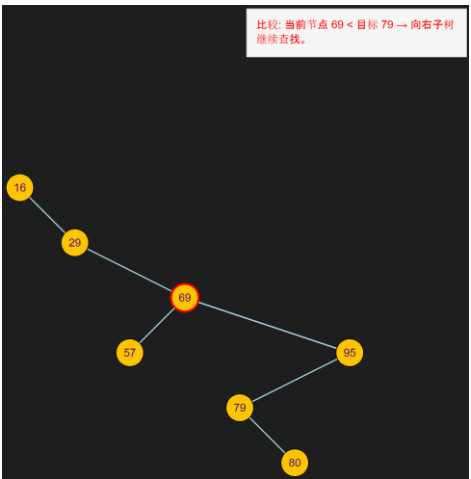
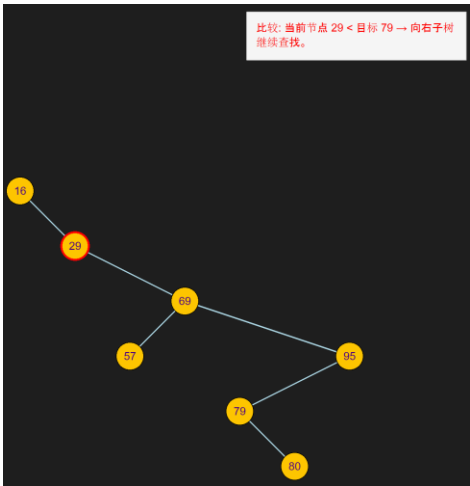
预期结果：画布上应生成正确的二叉树结构。查找过程中，系统应逐步高亮从根节点到目标节点的路径，提示区实时显示“比较：当前节点 > 目标... 向左查找”等逻辑判断。

测试结果：如图所示，查找时当前被标记为红色，逻辑判断准确。（此时随即生成的序列为 16,29,69,57,95,79,80;）

下图是绘制结果



下 4 图是查找节点 79 的过程

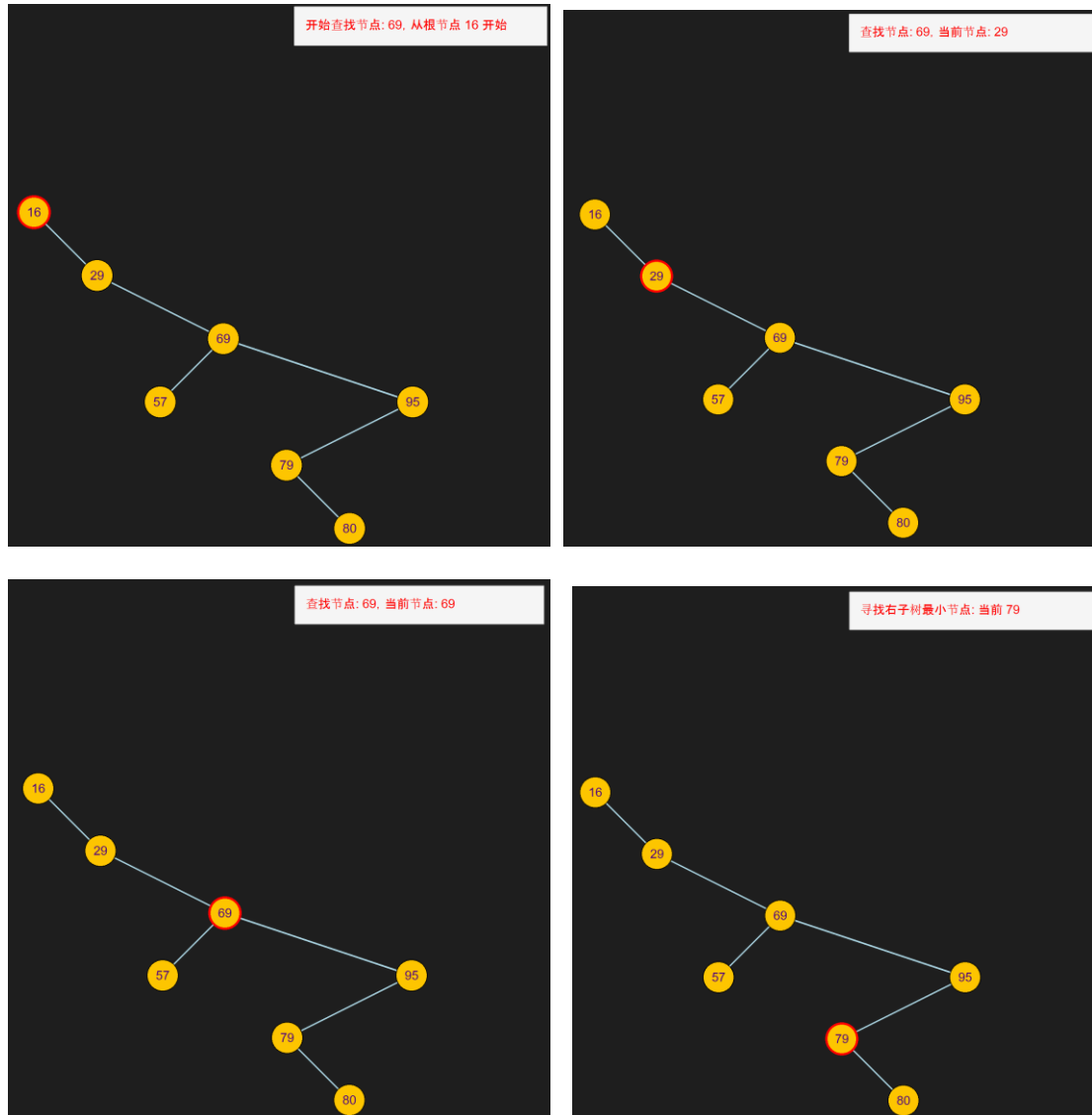


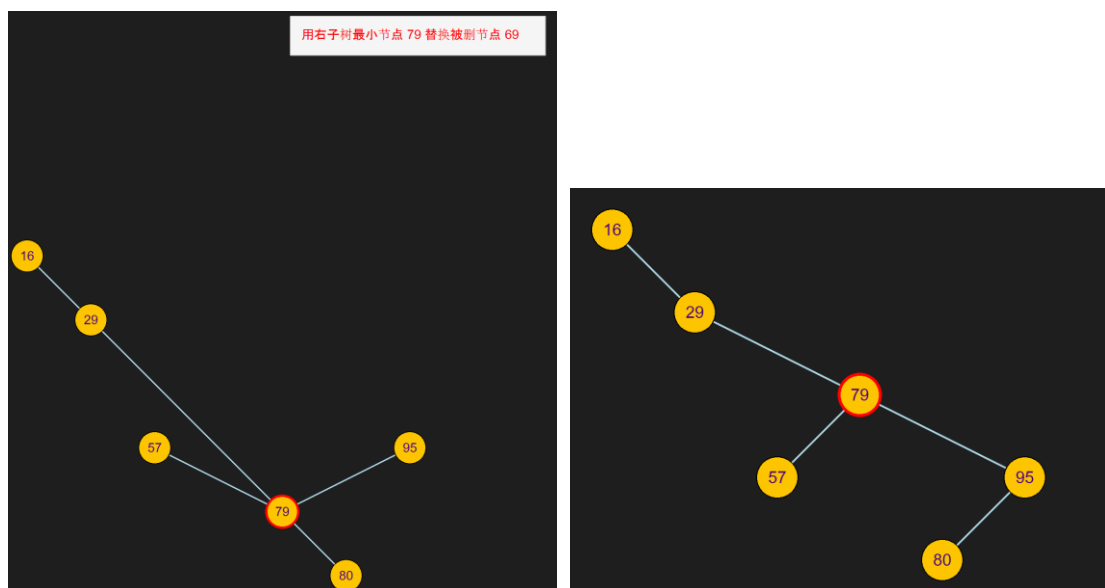
测试用例 4：BST 删除双子节点

操作：在 BST 中删除一个同时拥有左右子树的节点。

预期结果：系统应演示“寻找右子树最小节点（后继）”的过程，高亮后继节点，将其值替换到待删节点位置，最后删除原后继节点。

测试结果：如图 5.4 所示，删除过程符合 BST 算法逻辑，树结构保持有序。删除 69 节点。





5.1.3 AVL 树自平衡测试

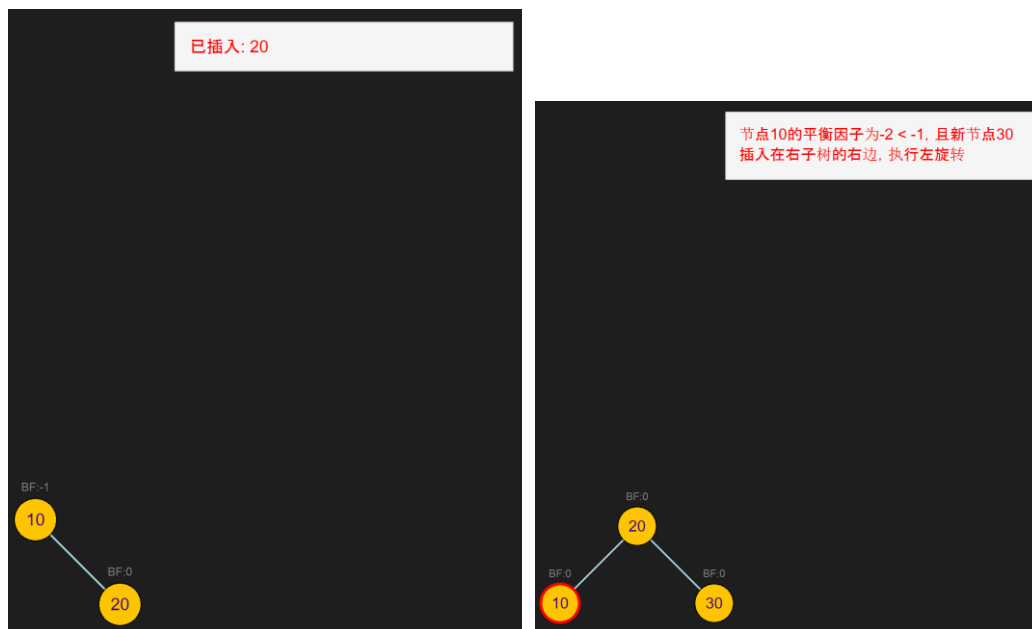
测试目标：验证 AVL 树在插入导致失衡时，能否自动触发正确的旋转（LL/RR/LR/RL）。

测试用例 5：RR 型失衡与左旋

操作：选择“AVL 树”模式，依次插入 10, 20, 30。

预期结果：插入 30 后，节点 10 的平衡因子变为 -2（右偏），系统应识别为 RR 型失衡，并在提示区显示“执行左旋转”。旋转后 20 应成为新的根节点，10 为左子，30 为右子，全树平衡。

测试结果：如图 4.5 所示，树结构自动调整为平衡状态，旋转类型判断正确。



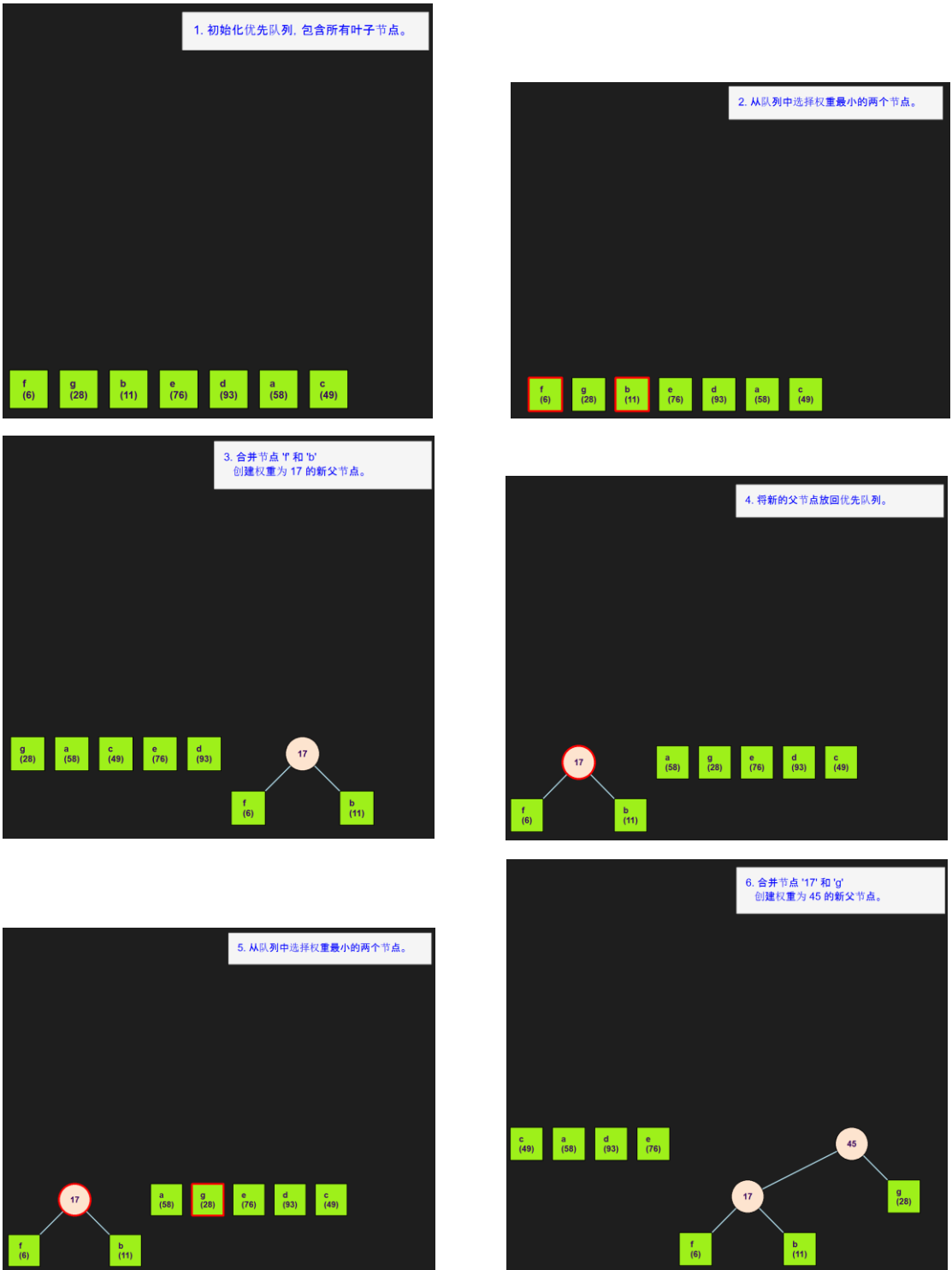
5.1.4 哈夫曼树与 B-树测试

测试用例 6：哈夫曼树构建

操作：选择“哈夫曼树”模式，点击“随机生成”获得字符权重，点击“开始演示”并连续点击“下一步”。

预期结果：系统应展示森林中权重最小的两个节点合并生成新父节点的过程。最终生成一棵哈夫曼树，并在右上角显示每个字符的哈夫曼编码。

测试结果：如图 4.6 所示，最小权重优先合并逻辑正确，编码符合前缀码特性。（序列为 e:76, a:58, g:28, b:11, d:93, f:6, c:49）



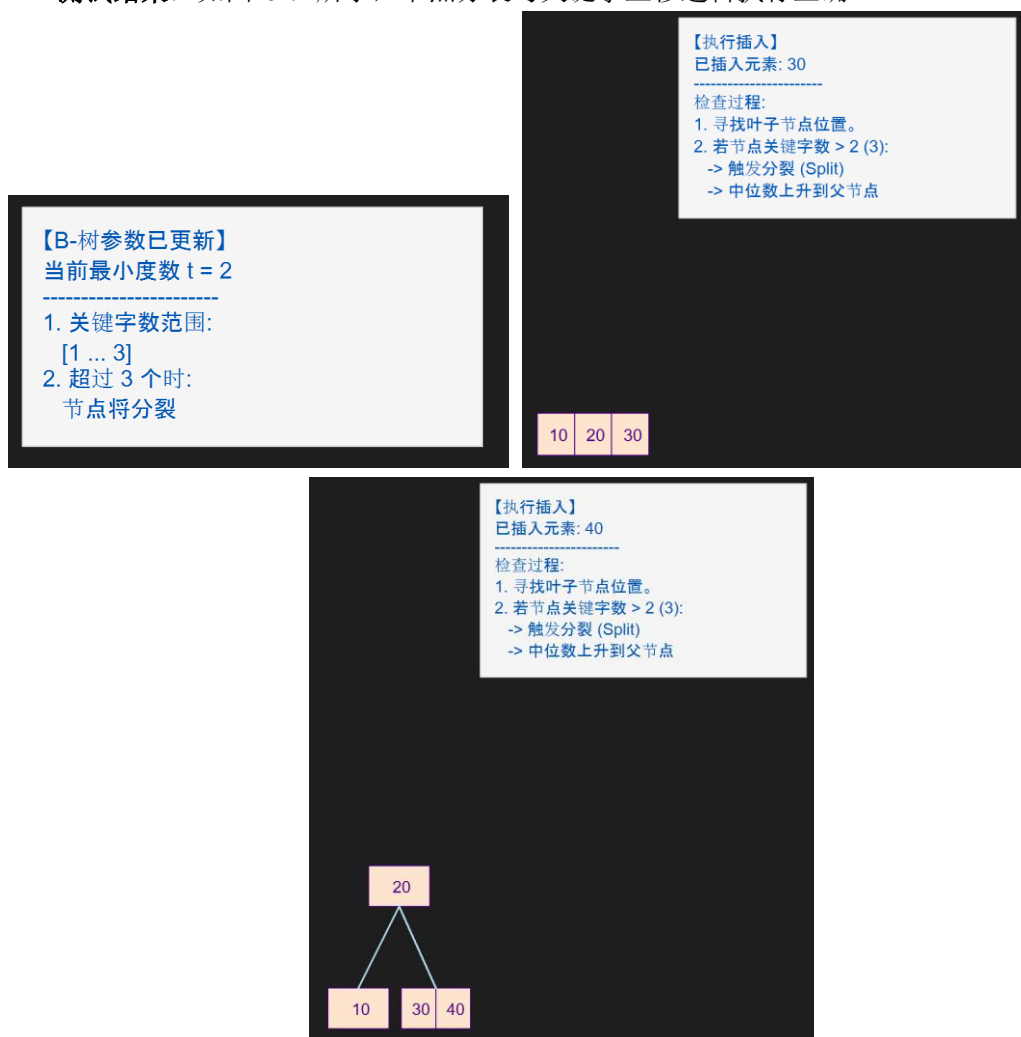
(后略)

测试用例 7：B-树节点分裂

操作：选择“B-树”模式，设置阶数 $t=2$ （节点最大关键字 3）。连续插入 10, 20, 30, 40。

预期结果: 当插入 40 时, 原根节点 [10,20,30] 已满, 应触发分裂。中间关键字 20 上升为新根, 10 和 30,40 分别位于两个子节点。

测试结果: 如图 5.7 所示, 节点分裂与关键字上移逻辑执行正确。



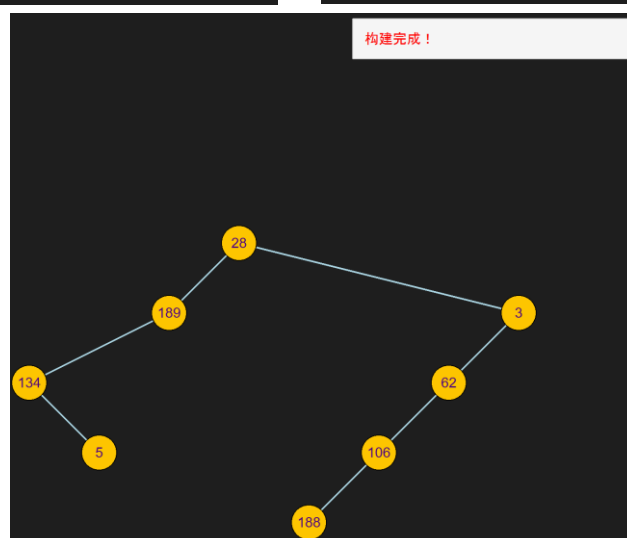
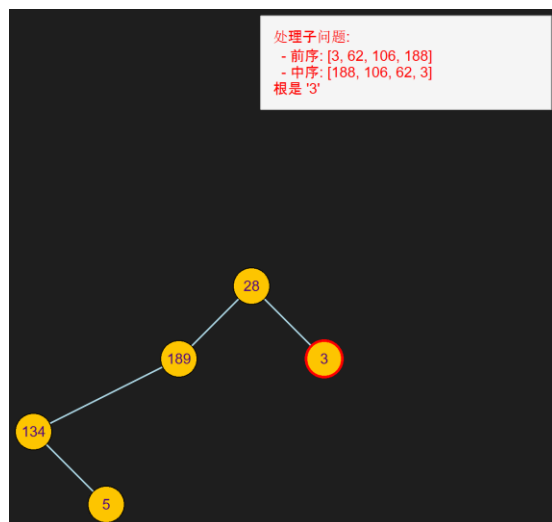
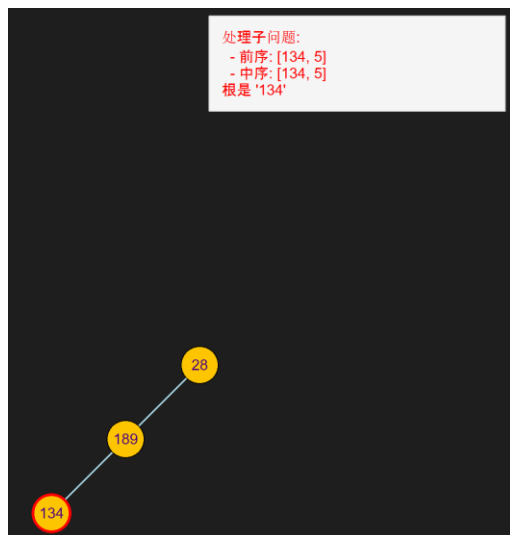
5. 1.5 普通二叉树演示测试

测试目标: 验证二叉树生成器驱动算法分步演示功能。测试两种 DFT (此处测试其中一种, 前序+中序序列构建), 一种 BFT 的二叉树创建功能。

测试用例 8: 前序+中序序列构建普通二叉树

操作: 随机生成 DFT 前序+中序序列 (此处为前序 28,189,134,5,3,62,106,188; 中序 134,5,189,28,188,106,62,3)

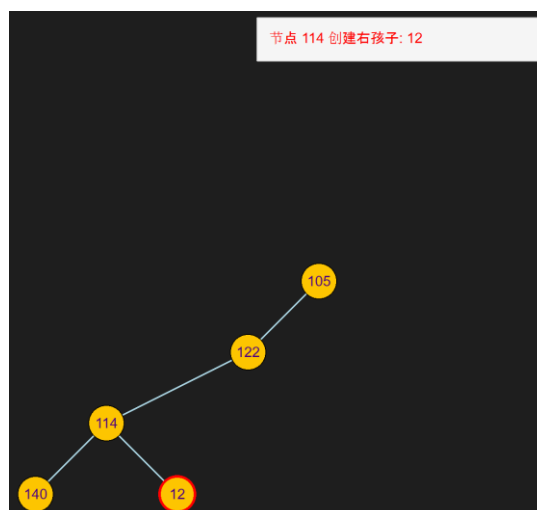
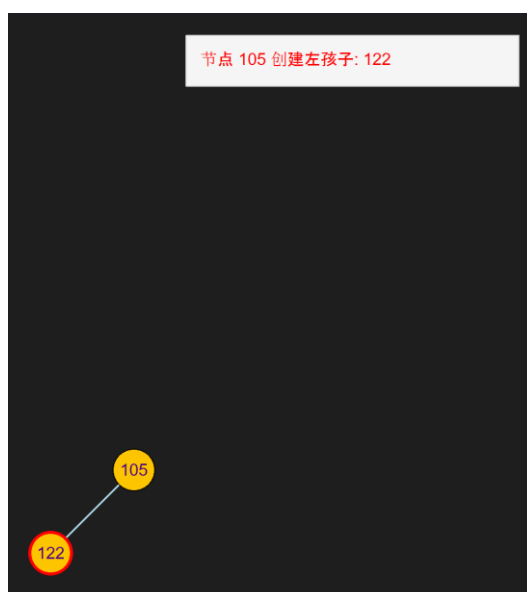
此处展示其中一些分步演示的步骤



测试用例 9: BFT 序列构建普通二叉树

操作: 随机生成 BFT 序列 (此处为 105,122,null,114,null,140,12)

此处展示其中一些分步演示的步骤



5.1.6 算法动态演示测试

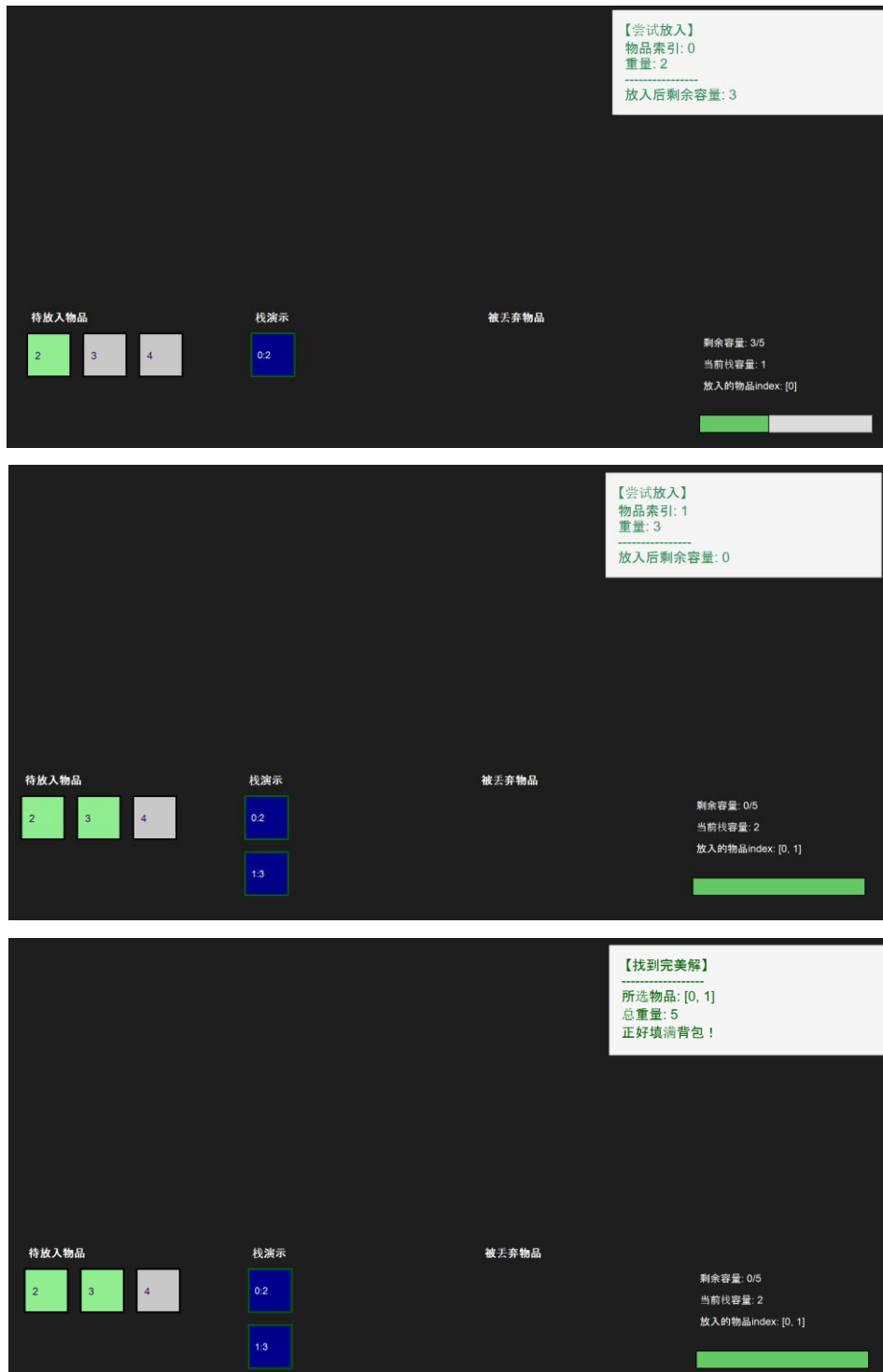
测试目标：验证生成器驱动算法分步演示功能。

测试用例 10：背包问题回溯

操作：选择“算法演示”->“背包问题”。输入物品 2,3,4，容量 5。点击“运行”并连续点击“下一步”。

预期结果：系统应展示物品入栈（尝试放入）与出栈（回溯）的过程。当栈中物品为 2, 3 时，总重 5，提示“找到完美解”并变绿。

测试结果：如图 5.8 所示，回溯路径清晰，成功找到最优解。



测试用例 11：中缀转后缀与求值

操作：选择“表达式求值”，输入 $(3+4)*5$ ，运行转换演示；完成后运行求值演示。

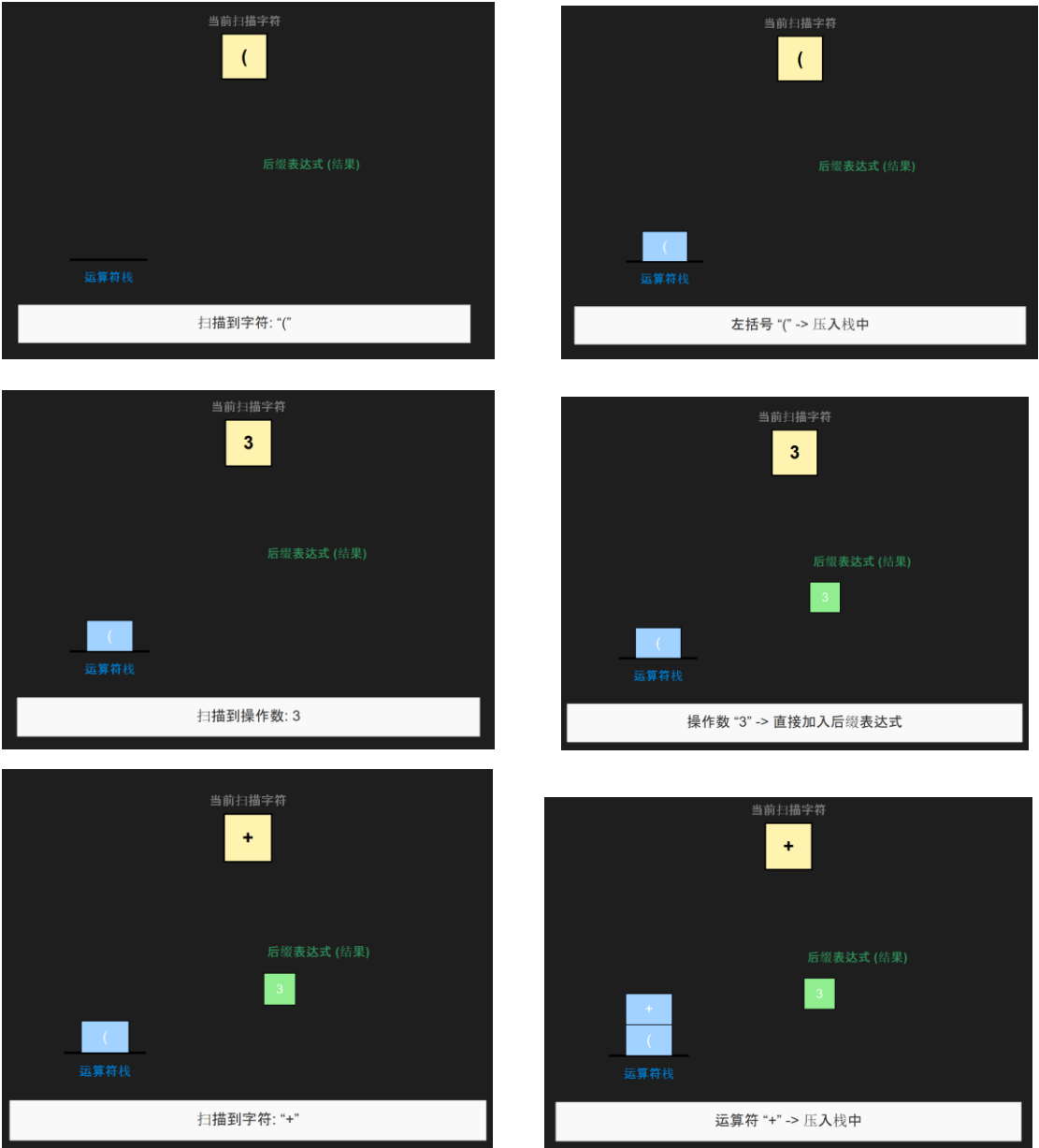
预期结果：

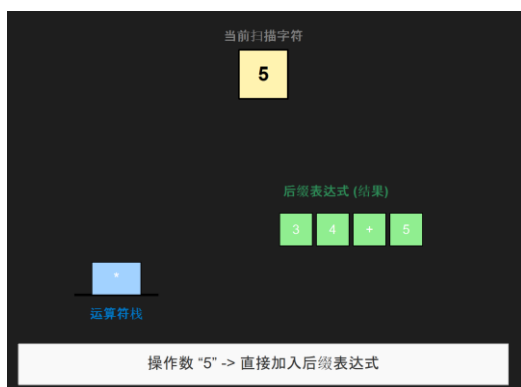
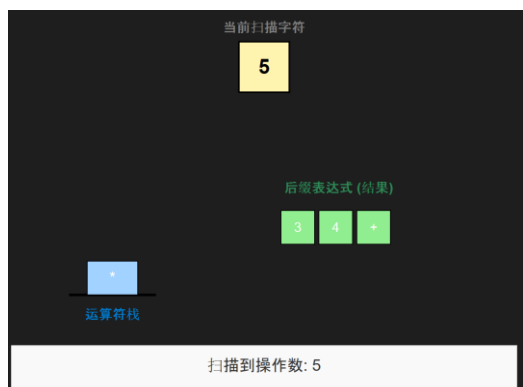
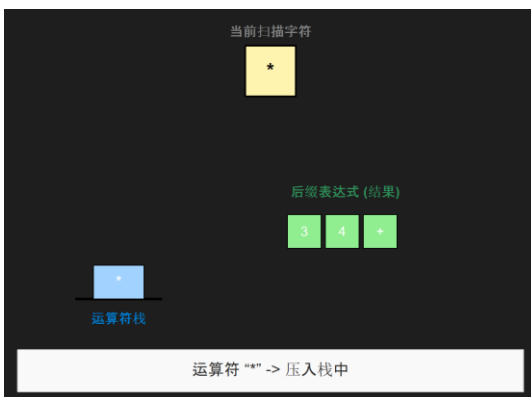
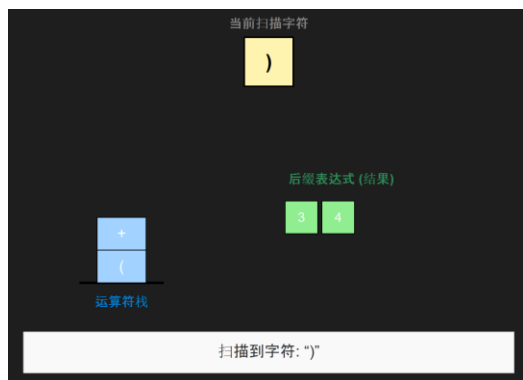
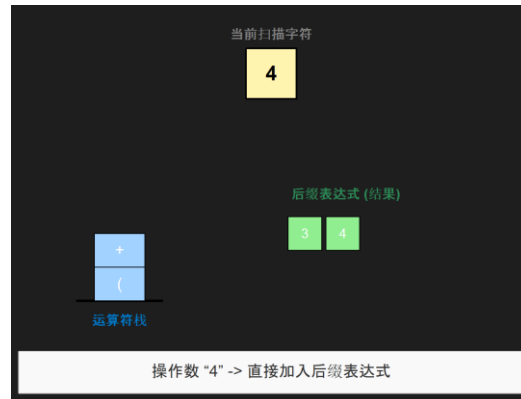
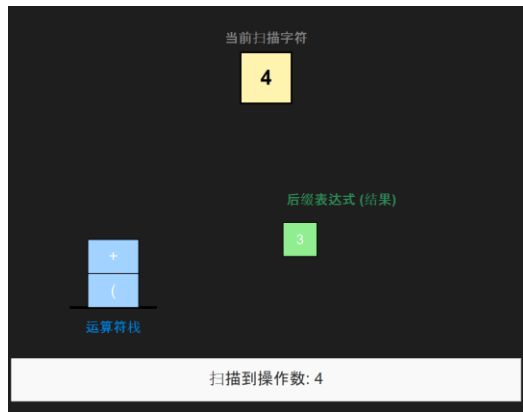
转换过程：栈的变化应展示 $($ 和 $+$ 的入栈出栈，最终输出 $3\ 4\ +\ 5\ *$ 。

求值过程：先计算 $3+4=7$ ，再计算 $7*5=35$ 。

测试结果：如图 5.9 所示，转换结果正确，最终计算结果为 35.0。

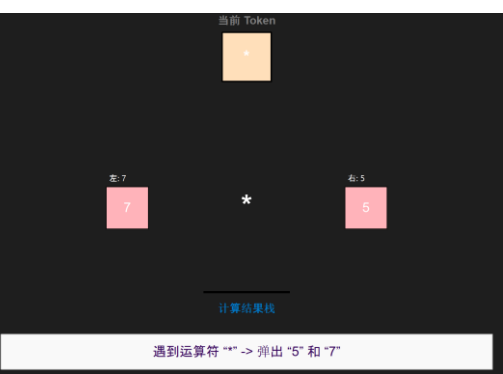
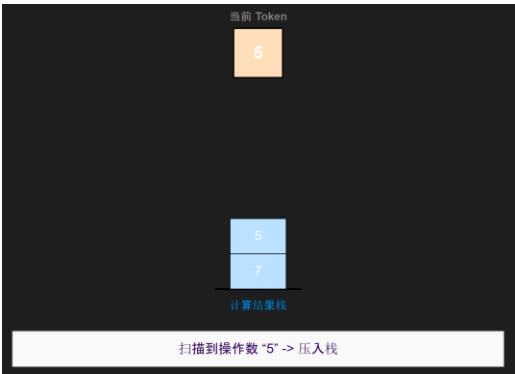
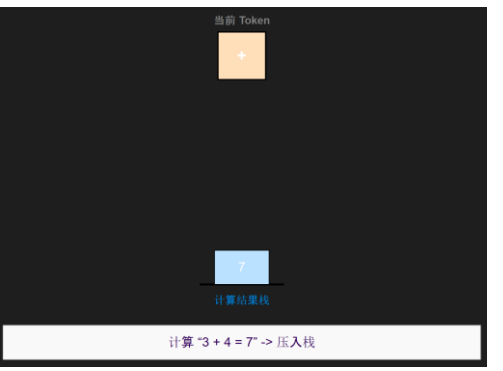
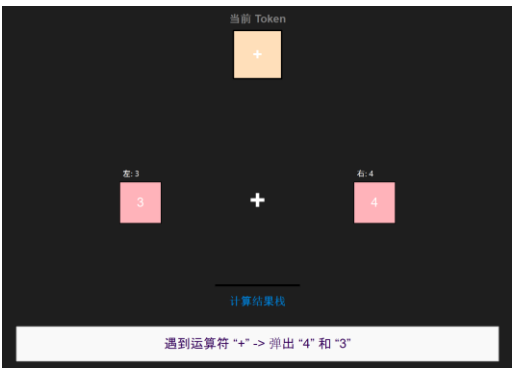
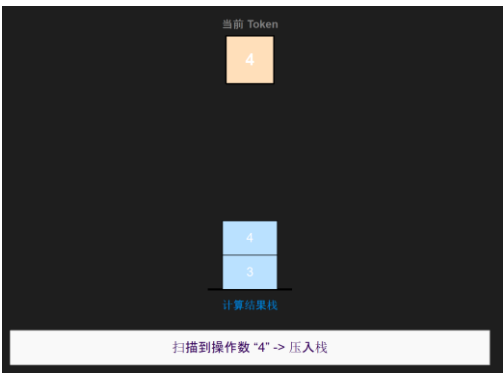
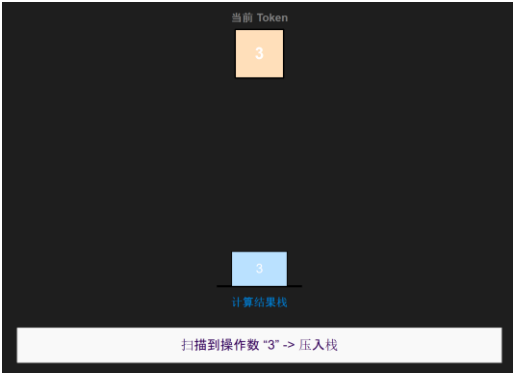
以下为求后缀表达式过程：

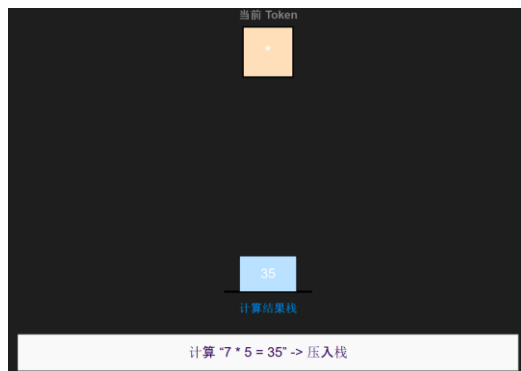






以下为求值展示：





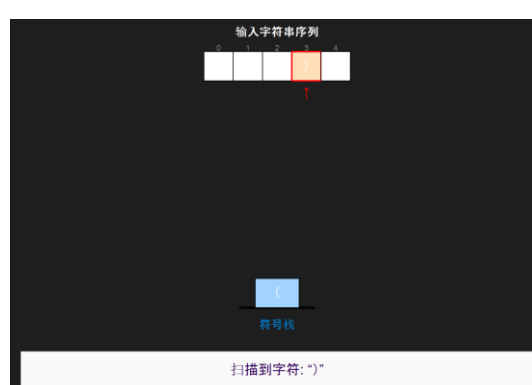
5.1.7 括号匹配问题测试

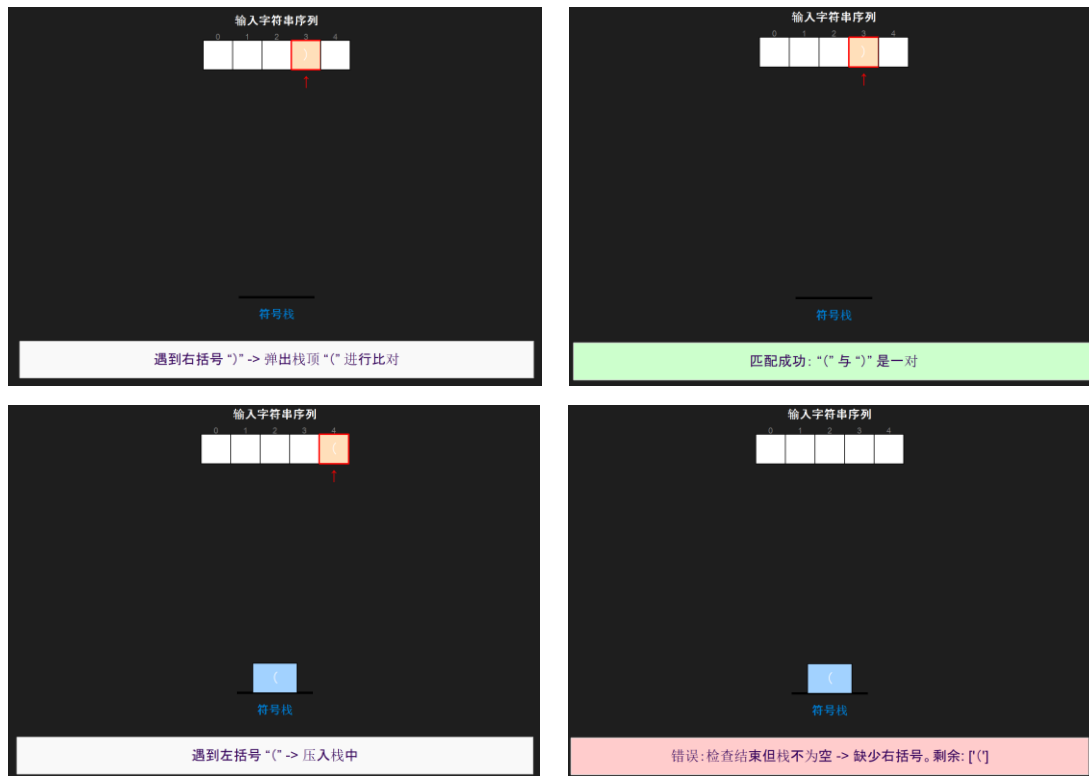
测试用例 12：括号匹配

操作：在文本框中输入括号序列 “{ } () (”

预期结果：系统识别出不匹配 “(”

测试结果：如图 5.10 所示，成功演示了不匹配的括号





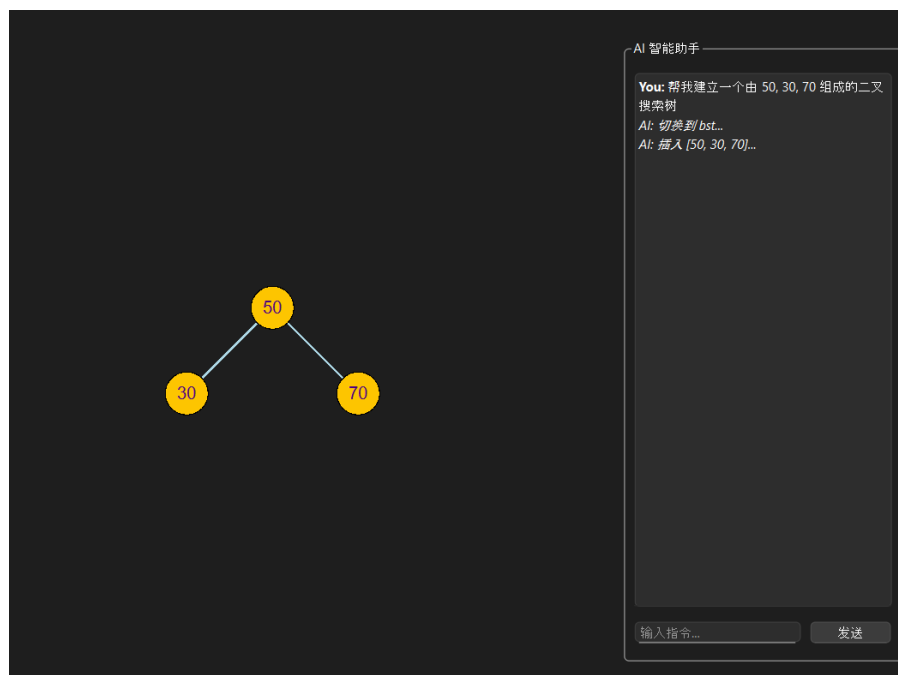
5.1.8 AI 智能交互测试

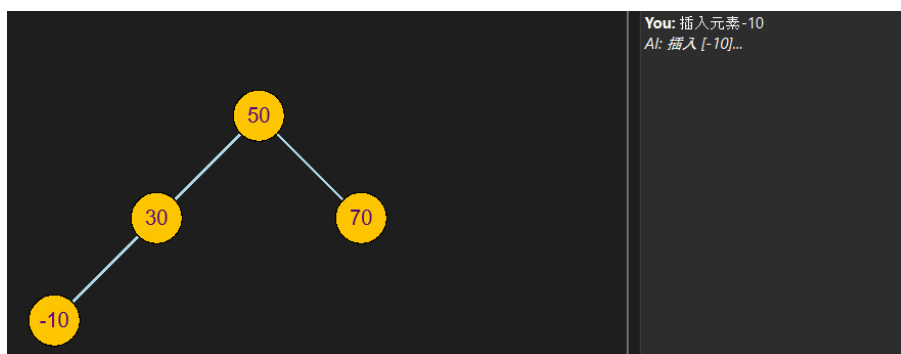
测试用例 13：自然语言指令控制

操作：在右侧 AI 助手框输入：“帮我建立一个由 50,30,70 组成的二叉搜索树”，点击发送。随后，发送“插入元素-10”

预期结果：系统自动识别意图，切换到 BST 模式，并依次插入 50,30,70，最终画布上显示正确的树结构。随后，能够有效插入元素

测试结果：如图 5.10 所示，AI 成功解析指令并调用了后端接口完成绘图。





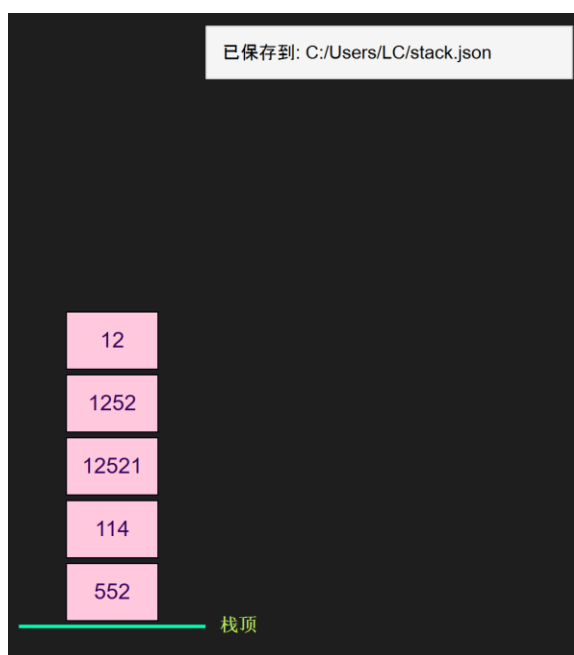
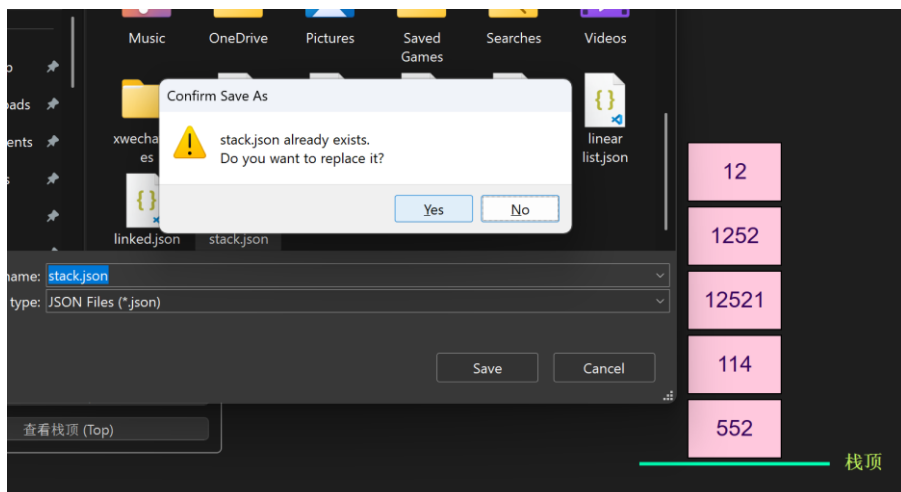
5.1.9 数据结构保存、读取测试

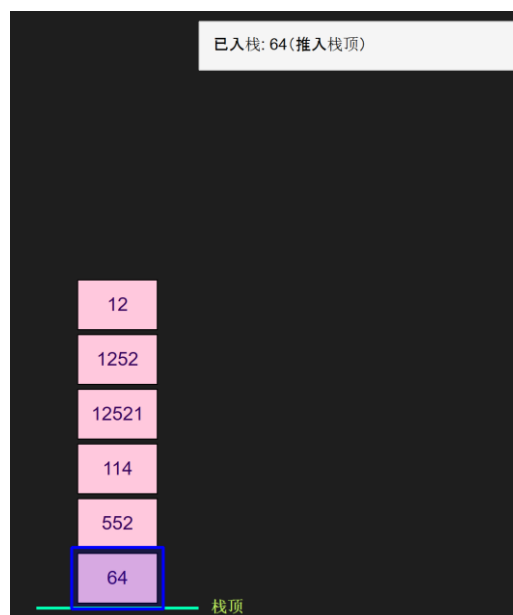
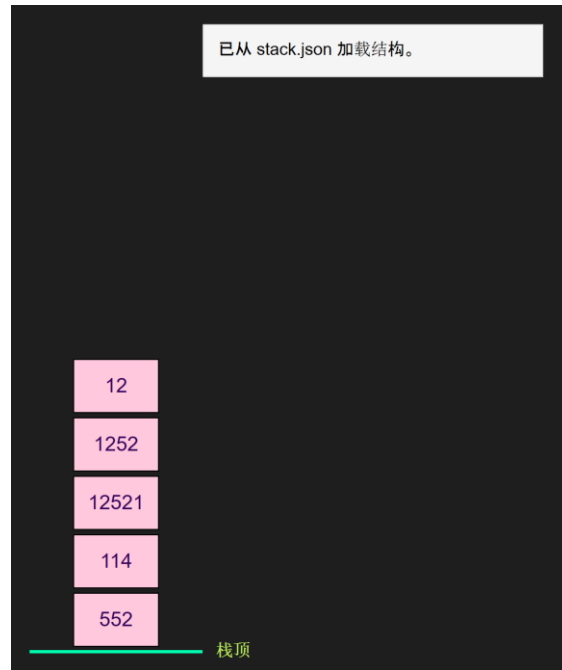
测试用例 14：数据结构保存、读取

操作：绘制数据结构后，点击“保存”到本地；随后，从别的数据结构模式下，点击“加载”。此外，再进行操作（如插入一个元素）

预期结果：系统能够正常保存、加载数据结构，并能进行元素的插入（基本操作）

测试结果：如图 5.10 所示，系统正常保存、加载数据结构，并进行了元素的插入。





5.2 错误用例

5.2.1 非法输入测试

测试用例：在数值输入框中输入非数字字符（如 "a"）并尝试插入。

处理逻辑：MainController 中的 handle_insert 函数包含 try...except ValueError 捕获块。

测试结果：程序未崩溃，控制台或状态栏忽略了非法输入，保持当前状态不变，系统具有鲁棒性。



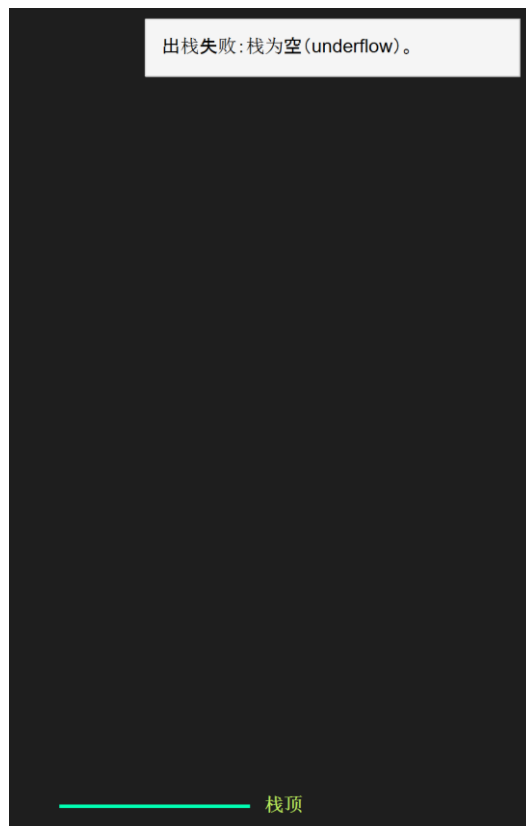
5.2.2 逻辑违规操作

测试用例 1：空栈出栈

操作：在栈为空时，点击“出栈”按钮。

预期结果：后端 `SequentialStack.pop()` 返回 `None`，前端捕获后在提示区显示错误信息。

测试结果：如图 5.11 所示，提示区显示“出栈失败：栈为空（underflow）”，程序正常运行。

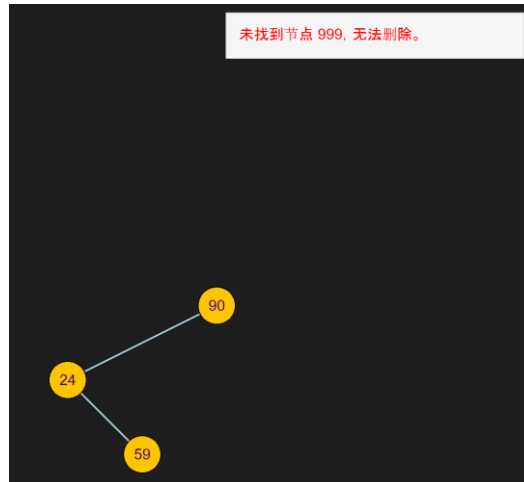


测试用例 2：删除不存在的节点

操作：在 BST 中尝试删除一个不存在的数值（999）。

预期结果：delete_path 生成器遍历全树后未能匹配 key，返回提示信息。

测试结果：提示区显示“未找到节点 999，无法删除”，树结构未发生改变。

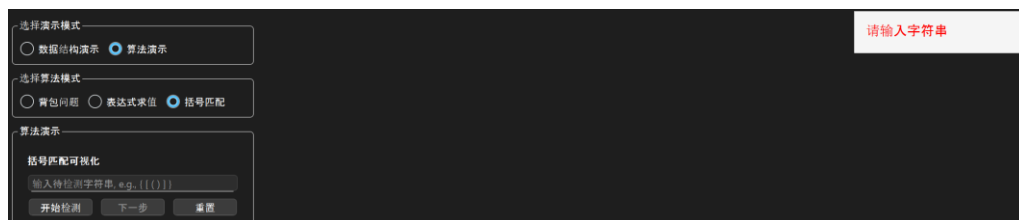


5.3 极端情况测试

5.3.1 空数据处理

测试用例：在没有任何数据的情况下，点击“检测括号序列”。

测试结果：未引发 NullPointerException 或 IndexError 异常。输出“请输入字符串”提示



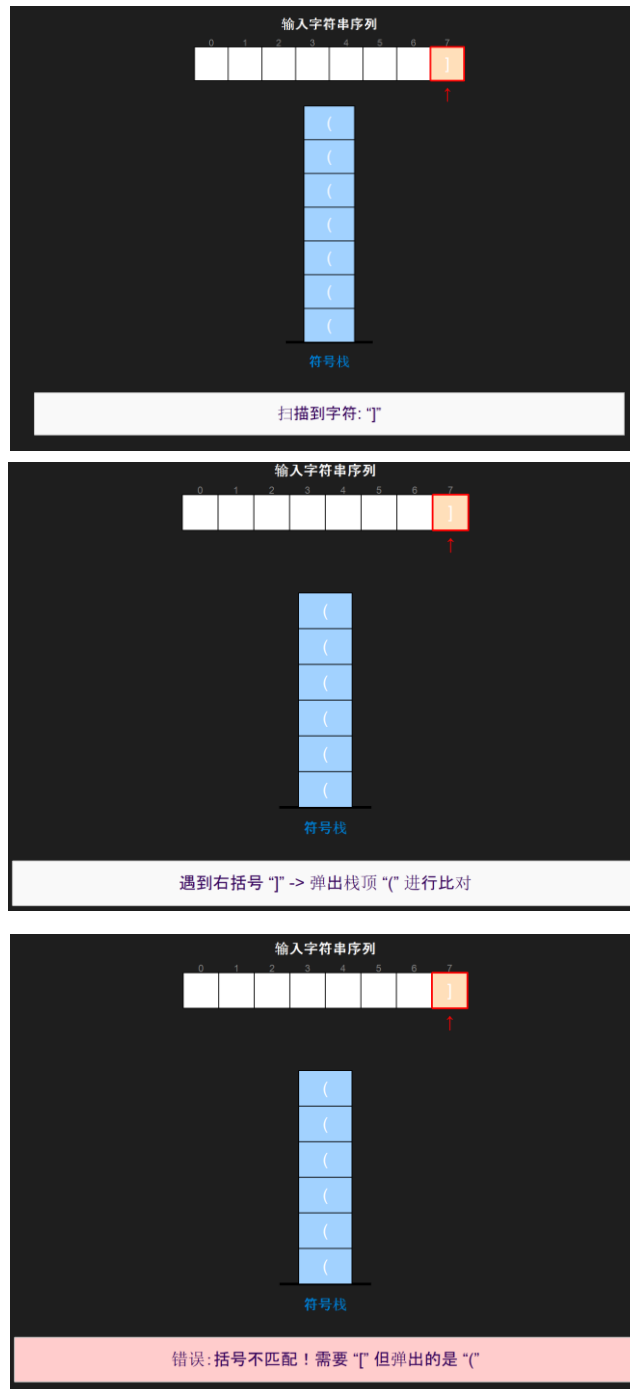
5.3.2 复杂边界情况

测试用例：括号匹配嵌套深度

操作：输入多重嵌套且不匹配的字符串 (((((([]).

预期结果：算法应能正确处理深层入栈，并在遇到] 时检测到栈顶为 (，从而报错“括号不匹配”。

测试结果：系统正确在合适的字符处停止



6. 总结与提高

6.1 体会与收获

本次课程设计不仅是一次编程实践，更是一次将理论知识转化为实际应用能力的综合训练。通过开发这款数据结构可视化演示系统，我在以下几个方面收获颇多：

1. **Python 语言实践：**这是我第一次使用 Python 语言完成完整的课程设计项目。在开发过程中，我深刻体会到了 Python 语法简洁、库丰富的特点，这种便捷性极大地提高了我的开发效率，让我能将更多精力集中在算法逻辑而非繁琐的语法细节上。

2. **独立算法实现：**在核心算法（如 AVL 树旋转、B-树分裂、背包问题回溯）的编写过程中，我坚持第一次不参考网络现成代码，而是完全依据课本原理和逻辑图进行自主编写与调试。这一过程虽然曲折，但极大地加深了我对数据结构底层原理的理解。
3. **快速开发体验：**鉴于本学期学业繁重，本次开发采用了敏捷开发的思路。相较于之前的 C 语言课设，利用 Python 和 PySide6 框架，我在较短的时间内就构建出了功能完善的原型，体验到了现代高级语言的巨大优势。
4. **GUI 框架掌握：**通过学习和使用 Qt (PySide6) 框架，我掌握了 GUI 绘制的方法，并体验了 Qt Designer 在界面布局设计上的便捷性。这让我认识到，良好的交互设计对于提升软件的用户体验至关重要。
5. **算法设计与优化：**在编写前端可视化算法时，由于初期规划不足，曾导致代码冗余和逻辑混乱，进而产生了潜在的 Bug。在后期的重构与修复过程中，我采用了剪枝等优化策略，将性能损耗降至最低。这次经历让我深刻认识到“设计先行”的重要性——在编码前进行周密的规划和文档编写，能有效减少后期的返工成本。
6. **文档撰写的磨砺：**撰写这份报告是一项艰巨的任务。由于本学期选课较多，有许多课程报告，时间非常紧迫。我认为，认真撰写报告不仅是对自己劳动成果的尊重，更是对耐心与学术严谨性的锻炼。面对未来科研道路上的挑战，这种沉下心来做好每一件事的态度是必不可少的。

6.2 问题与解决

在开发过程中，我遇到了不少挑战，主要集中在前端函数编写与后端数据同步上。针对这些问题，我采取了以下解决方案：

语法与框架生疏：

*问题：*由于 Python 语法遗忘和对 Qt 机制不熟悉，初期开发进度缓慢。

*解决：*我利用 test.ipynb 文件建立了一个独立的测试环境，通过编写简单的 print 验证代码来重温语法特性；同时查阅 Qt 官方文档，对照示例代码进行调试，逐步掌握了信号与槽的使用方法。

特例考虑不周：

*问题：*在处理如“删除不存在的节点”或“空栈出栈”等边界情况时，程序曾多次崩溃。

*解决：*我采用了“追根溯源”的调试方法，通过在关键路径插入日志打印，观察后台数据的实时变化，从而定位到逻辑漏洞，并针对性地添加了 try-except 和边界检查代码。

数据同步错误：

*问题：*在可视化过程中，有时会出现界面显示与后台数据不一致（如栈已空但界面仍有元素残留）的情况。

*解决：*我坚持在每次 UI 更新前强制同步后端快照，并利用 print 方法时刻监控后端数据流。一旦发现视图与模型状态偏差，立即检查 Controller 层的逻辑，最终修复了多处数据更新不同步的问题。

6.3 自我评估

亮点：

我认为我的程序在**交互性**和**鲁棒性**方面表现出色。系统不仅实现了基础的增删查改，还考虑了大量的特殊情况（如非法输入、空操作），并提供了详尽的文字反馈。

额外完成了**自然语言交互（AI 助手）**和**分步算法演示**等扩展功能，超出了题目的基

本要求。

不足：

主要在于**前端算法的架构设计**。由于开发初期缺乏详细的规划，导致部分视图更新逻辑（如重绘整棵树而非局部刷新）存在冗余，虽然在当前数据量下未造成卡顿，但在处理大规模数据时可能会有性能瓶颈。

具体来说，交互时需要频繁通过全局变量或深拷贝来传递图与课表信息（类比到本项目即树结构快照）。为了不破坏原有结构支持“回退”功能，我不得不进行大量的深拷贝操作。尽管我尝试了剪枝优化，但这本质上是架构设计层面的妥协。

这是一个宝贵的教训。在未来的开发中，我会吸取本次课设的经验，更加重视前期的架构设计与规划，力求写出更高效、更优雅的代码！